

Gerenciar a configuração e versionamento de Software

Jean Max Simon Krebs

03/10/2025

Rede
Fecomércio RS
de Educação



Controle de versão - GIT

Rede
Fecomércio RS
de Educação



Controle de versão - GIT



Git

Sistema de controle
de versão distribuído

Permite acompanhar
evolução de arquivos

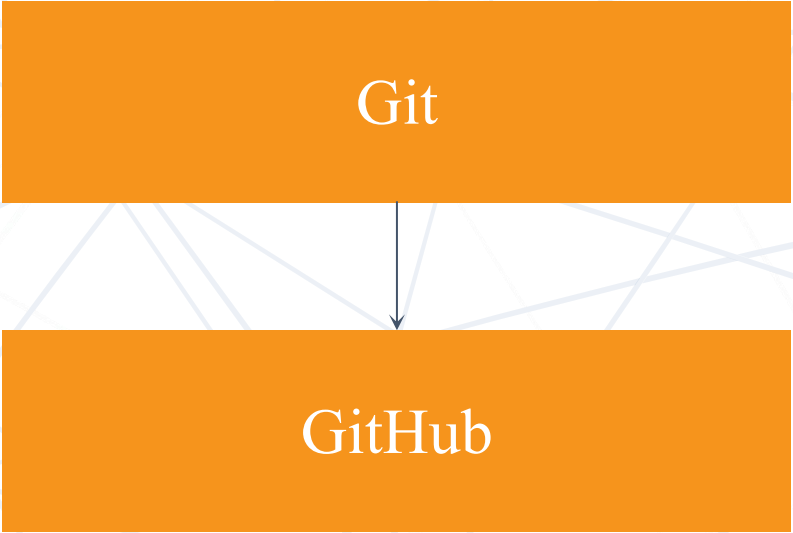
Git

Trabalhar de forma
segura sem perder
versões

Histórico de mudanças

Trabalhar em diferentes
funcionalidades ao
mesmo tempo

Criado por Linus Torvalds
(2005)



```
graph TD; Git[Git] --> GitHub[GitHub]
```

Git

GitHub

Git



GitHub

Plataforma online que
hospeda repositórios Git

Backup

Colaboração

Controle de acesso

Publicação de sites

Git

GitHub

Publicar site pessoal

Por que aprender tudo isso?



Por que aprender?

- Sem arquivos ZIP espalhados

Por que aprender?



Agora vai.zip



Em produção.zip



Em produção -
Corrigida.zip



Final.zip



Final mesmo.zip



Quase em
produção.zip

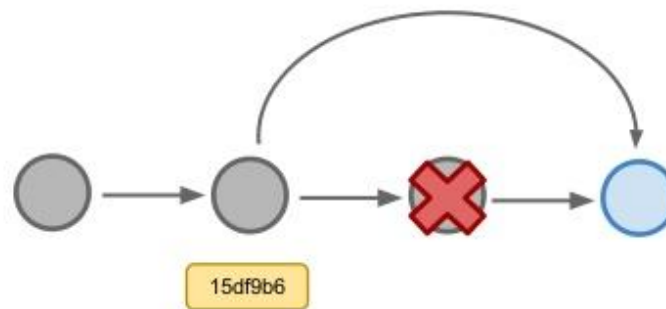
Por que aprender?

- Sem arquivos ZIP espalhados
- Poder voltar atrás

Por que aprender?

Example

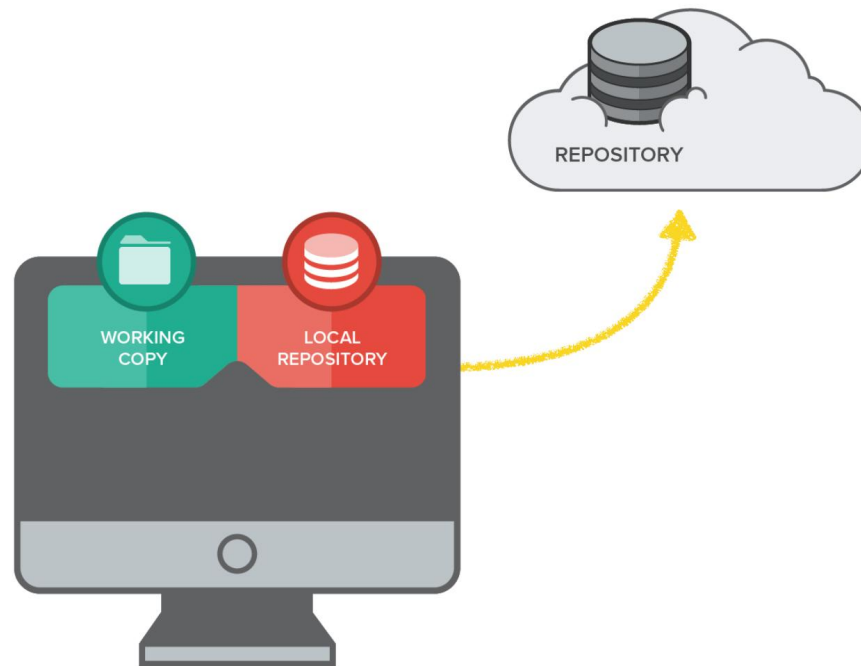
```
git commit -am "update readme"  
git revert 15df9b6
```



Por que aprender?

- Sem arquivos ZIP espalhados
- Poder voltar atrás
- Trabalho em equipe

Por que aprender?



Por que aprender?

- Sem arquivos ZIP espalhados
- Poder voltar atrás
- Trabalho em equipe
- Principal ferramenta do mercado (GIT)

Por que aprender?



Tech Lead

Tradesquash Barueri Pequena ou Média empresa Até
R\$12.000 Sênior PJ Aceita candidatos de fora

Python

PHP

Git

AWS S3

Por que aprender?

- Sem arquivos ZIP espalhados
- Poder voltar atrás
- Trabalho em equipe
- Principal ferramenta do mercado (GIT)
- Seu currículo online - GitHub

Por que aprender?

Overview Repositories 32 Projects 0 Packages 0 Stars 296 Followers 48 Following 36

Pinned Customize your pins

 **DotCEP**

Biblioteca para facilitar o trabalho com CEP, utilizando .NET C#

 C# ★ 22 🍴 11

 **BaltaStore**

Aulas do curso Criando APIs com ASP.NET Core 2.0 e Dapper

 JavaScript

 **Gatel**

Controle de acesso de veículos em residenciais utilizando OCR - Gatel

 C#

 **Arduino**

Códigos de estudo com arduino

 C++ ★ 1 🍴 1

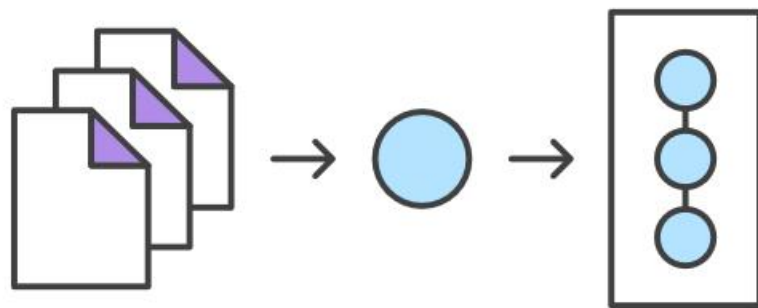


O que é Git?

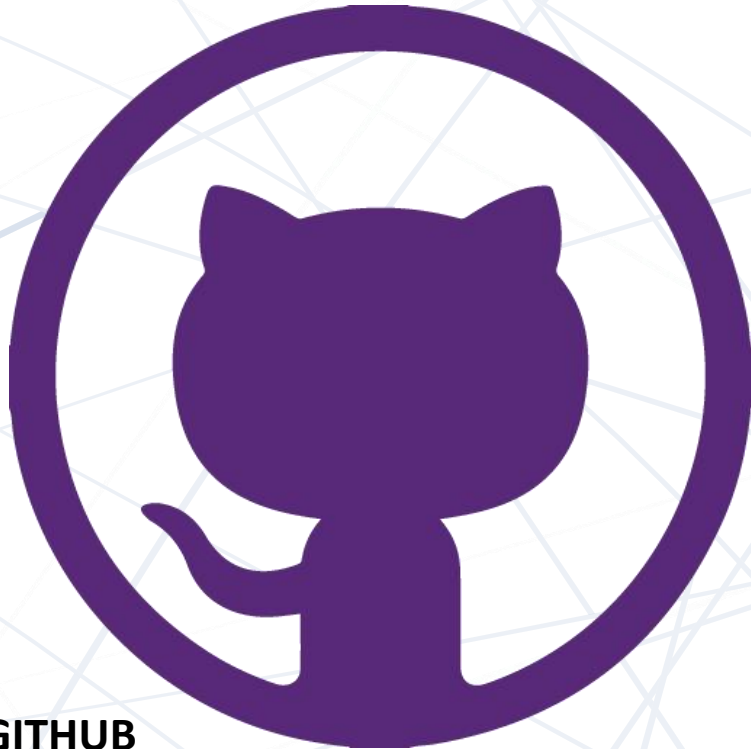
Git é um sistema de controle de versão de arquivos. Através deles podemos desenvolver projetos no qual diversas pessoas podem contribuir simultaneamente, editando e criando novos arquivos e permitindo que os mesmos possam existir sem o risco de suas alterações serem sobrescritas.

Por que aprender?

1. Mantendo o histórico do código
2. Trabalhando em equipe
3. Sistemas de controle de versão
4. Controle de versão rápido e confiável
5. Hospedando código no GitHub



Ferramentas



GITHUB



axosoft
GitKraken

O que é GitHub?



GitHub - Servidor Remoto



GitHub - Servidor Remoto

Meu Drive > Teste ▾



Pastas

Nome ↑

 Agora ta em produção

 Agora vai

 cadastro de casas feito

 Final mesmo

 Inicio

 login pronto

 Quase em produção

 Versão final

Alternativas



GOOGLE DRIVE



Dropbox



!=





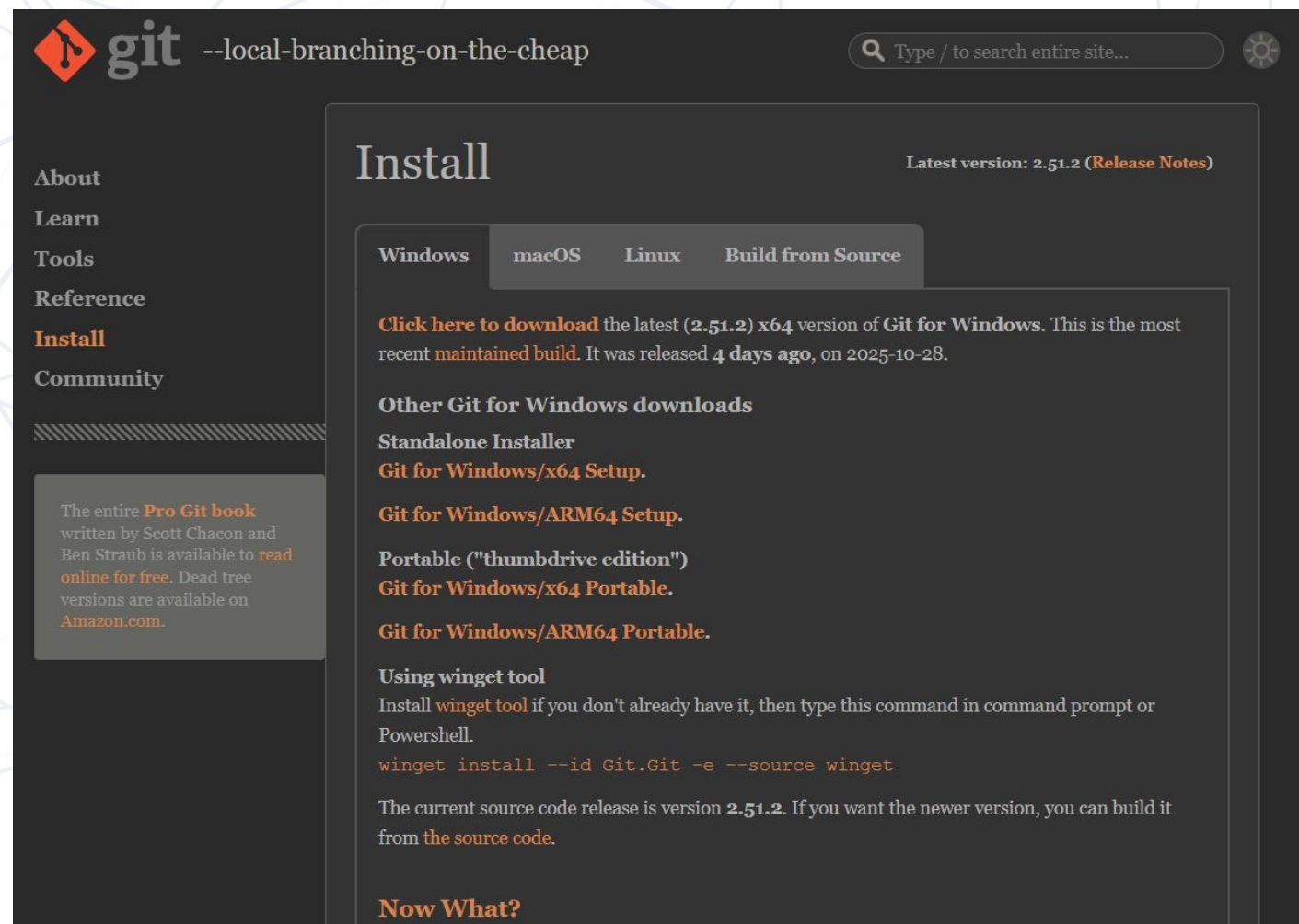
Etapa 0 - Instalação e Git init

Instalando Git

Baixe o instalador:

•Acesse:

<https://git-scm.com/download/win>



The screenshot shows the Git website's 'Install' page. The header features the Git logo, the tagline '--local-branching-on-the-cheap', a search bar, and a settings icon. The left sidebar contains links for 'About', 'Learn', 'Tools', 'Reference', 'Install' (highlighted), and 'Community'. Below these is a promotional box for the 'Pro Git book'. The main content area is titled 'Install' and includes a 'Latest version: 2.51.2 (Release Notes)' link. It has tabs for 'Windows', 'macOS', 'Linux', and 'Build from Source', with 'Windows' selected. The 'Windows' section provides instructions on how to download the latest version, lists other download options like 'Standalone Installer', 'Git for Windows/x64 Setup', 'Git for Windows/ARM64 Setup', 'Portable ("thumbdrive edition")', and 'Git for Windows/x64 Portable', and includes a section for 'Using winget tool' with a command prompt example. It also mentions the current source code release version and a 'Now What?' section.

git --local-branching-on-the-cheap

Type / to search entire site...

About
Learn
Tools
Reference
Install
Community

The entire **Pro Git book** written by Scott Chacon and Ben Straub is available to **read online for free**. Dead tree versions are available on **Amazon.com**.

Install

Latest version: 2.51.2 ([Release Notes](#))

Windows macOS Linux Build from Source

Click here to download the latest (2.51.2) x64 version of Git for Windows. This is the most recent **maintained build**. It was released **4 days ago**, on 2025-10-28.

Other Git for Windows downloads

Standalone Installer
Git for Windows/x64 Setup.
Git for Windows/ARM64 Setup.

Portable ("thumbdrive edition")
Git for Windows/x64 Portable.
Git for Windows/ARM64 Portable.

Using winget tool

Install **winget tool** if you don't already have it, then type this command in command prompt or Powershell.

```
winget install --id Git.Git -e --source winget
```

The current source code release is version 2.51.2. If you want the newer version, you can build it from **the source code**.

Now What?

Instalando Git

Abra o instalador (.exe) e avance nas telas:

- Clique em Next

Marque todas as caixas

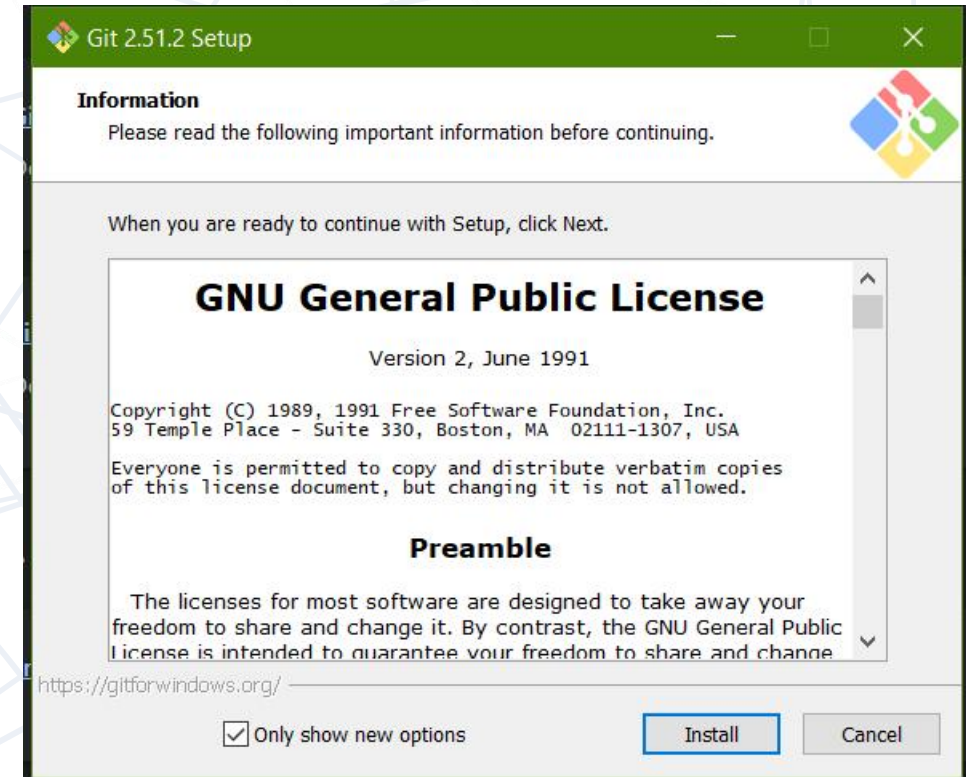
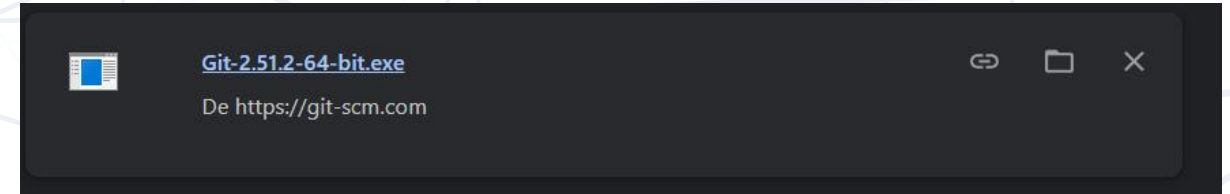
- Clique em Next

Quando aparecer a opção "Choosing the default editor used by Git", escolher a opção "Use visual studio code as git's default editor".

- Clique em Next duas vezes

Na parte "Adjusting your PATH environment", selecione "Git from the command line and also from 3rd-party software".

- Continue clicando em Next até Install.



Instalando Git

Marque a caixa “Launch Git Bash”

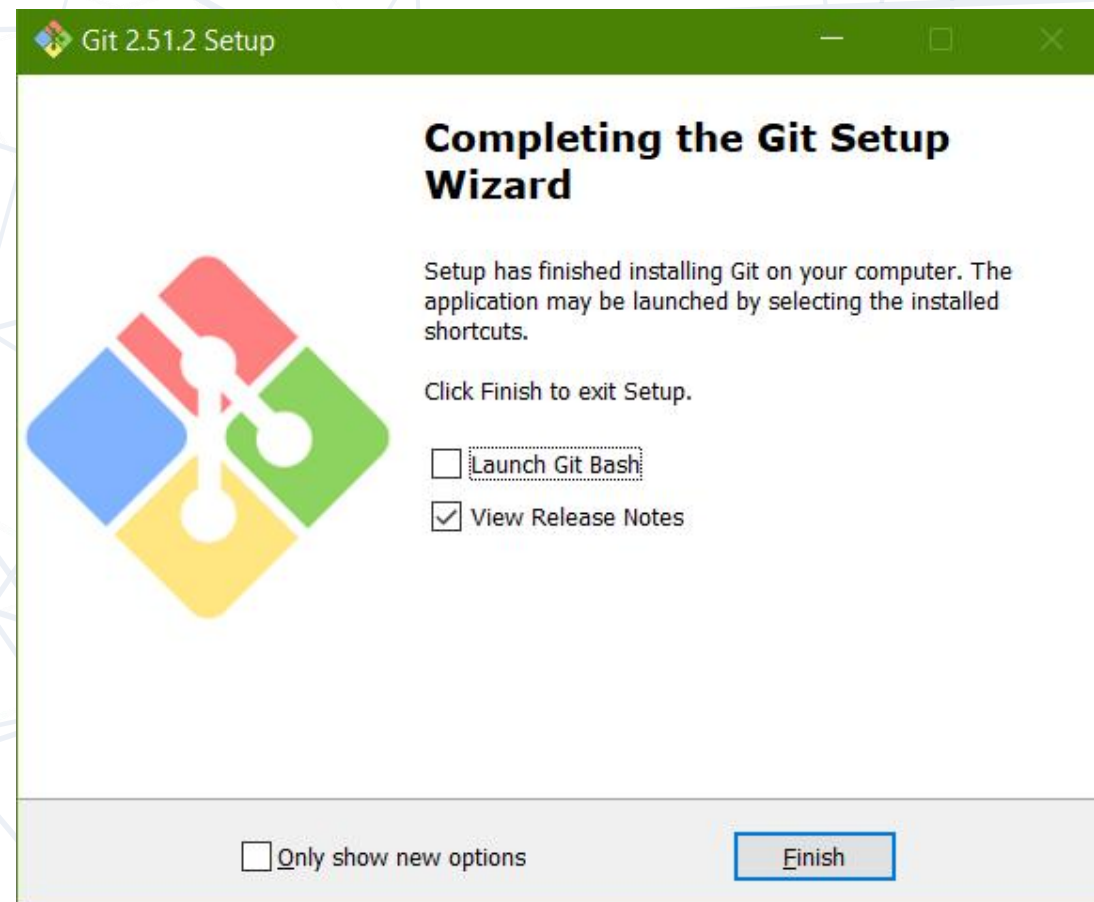
- Clique em Finish

Teste se instalou corretamente:

- Abra o Git Bash e digite:

`git --version`

```
tsije@DESKTOP-1KRUSLS MINGW64 ~  
$ git --version  
git version 2.51.2.windows.1
```



O que é o Git Bash

É um aplicativo que emula a linha de comando do Linux no Windows, permitindo executar comandos Git e outras utilidades de linha de comando diretamente no sistema operacional. Ele é uma ferramenta essencial para desenvolvedores que usam Windows e precisam interagir com o Git, um sistema de controle de versão, através de texto.

Utilizando Git Bash

Criando ambiente

Ir até o Desktop

- `cd ~/Desktop`

Criar uma pasta com seu nome

- `mkdir Nome_Sobrenome`

Entrar na pasta criada

- `cd Nome_Sobrenome`

Criar uma pasta chamada repositorio_portfolio_web

- `mkdir repositorio_portfolio_web`

Entrar nessa nova pasta

- `cd repositorio_portfolio_web`

Utilizando Git Bash

Iniciando Repositório Local

Iniciar o repositório Git dentro dessa pasta

- `git init`

Se tudo der certo, o Git Bash mostrará algo como:

“ Initialized empty Git repository in
C:/Users/seu_usuario/Desktop/Nome_Sobrenome/repositorio
_portfolio_web/.git/ “

Ir na Área de Trabalho, acessar a pasta e exibir arquivos ocultos para ver o arquivo `.git`

Configuração

Voltar ao git bash

defina seu nome e email

- `git config --global user.name 'Seu Nome Sobrenome'`
- `git config --global user.email 'Seu Email'`

Verifique

- `git config user.name`
- `git config user.email`
- `git config --global pull.rebase false`



Etapa 1 - Manipulação de arquivos

Git status

Verifica o status

Git status

Verifica o status

```
tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME (master)
$ git status
On branch master

No commits yet
```


Git status

- ~~Verifica o status~~
- Cria o arquivo

```
tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME (master)  
$ touch index.js
```

Exercício

Além da página index.html

Com os comandos já aprendidos, crie uma pasta chamada “css” e um novo arquivo chamado “style.css” dentro da pasta

E rode novamente o comando:

- git status

Git status

- ~~—Verifica o status~~
- ~~—Cria o arquivo~~
- Verifica o status

```
$ git status
On branch master

No commits yet

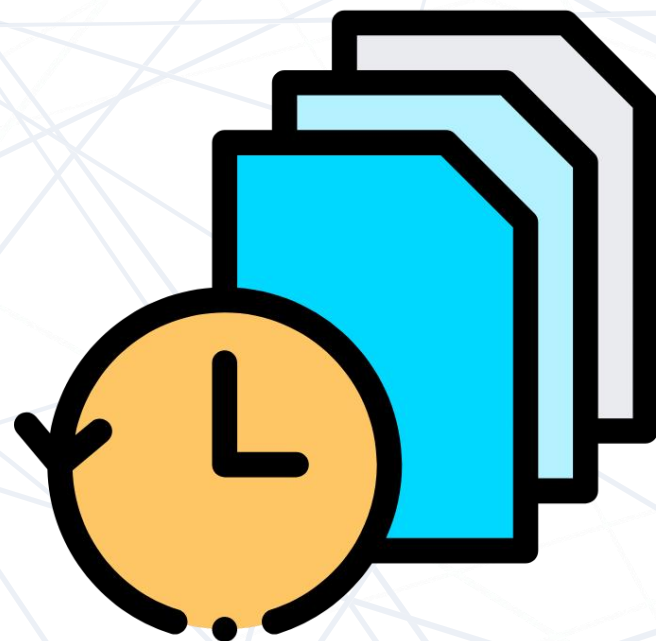
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    css/
    index.js

nothing added to commit but untracked files present (use "git add" to track)
```

Resultado esperado:

Arquivos vermelhos = ainda não rastreados “untracked”

Diretório .git já existe (não aparece, é oculto)

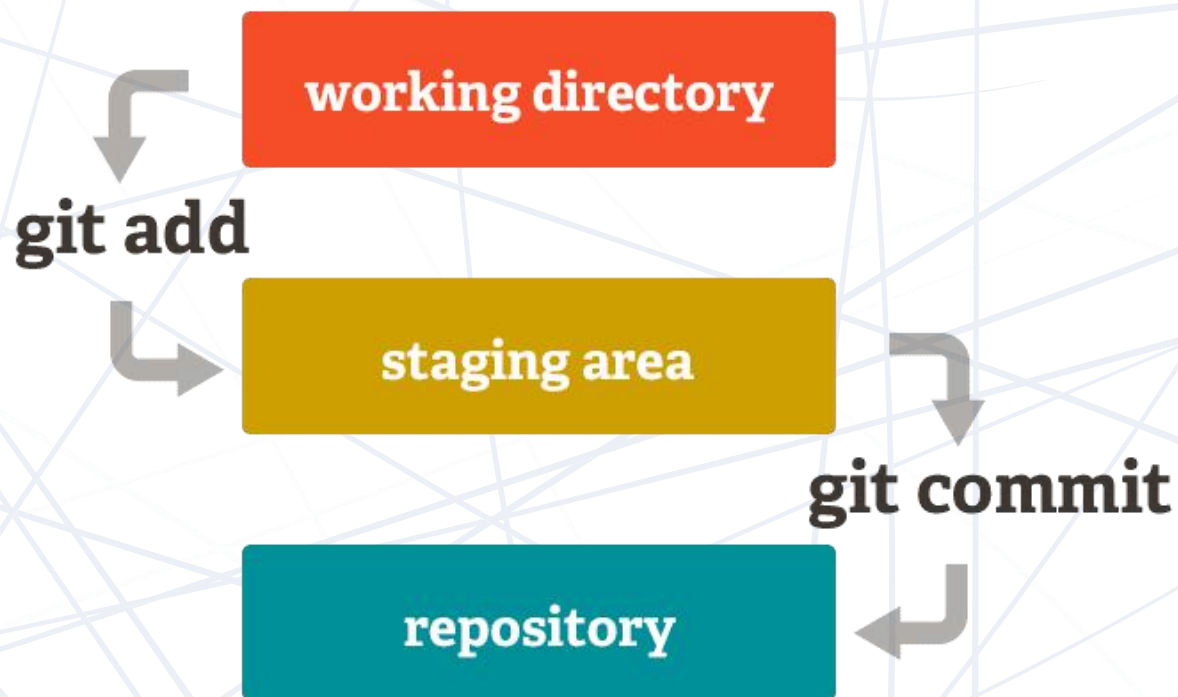


Etapa 2

Staging area - Área de Preparação

O que é

Um espaço intermediário onde você seleciona e agrupa as alterações que deseja incluir no próximo commit.



Staging area

Na janela de tempo entre uma versão e outra, os arquivos podem estar em quatro estados. Estes são:

- Não-rastreado (untracked)
- Não-modificado (unmodified)
- Modificado (modified)
- Staged

Estado Não-Rastreado (untracked)

Arquivos neste estado não estão presentes na versão atual e não constarão na próxima versão.

Estado Não-Modificado (unmodified)

Arquivos neste estado estão na versão atual e não foram alterados.

Estado Modificado (modified)

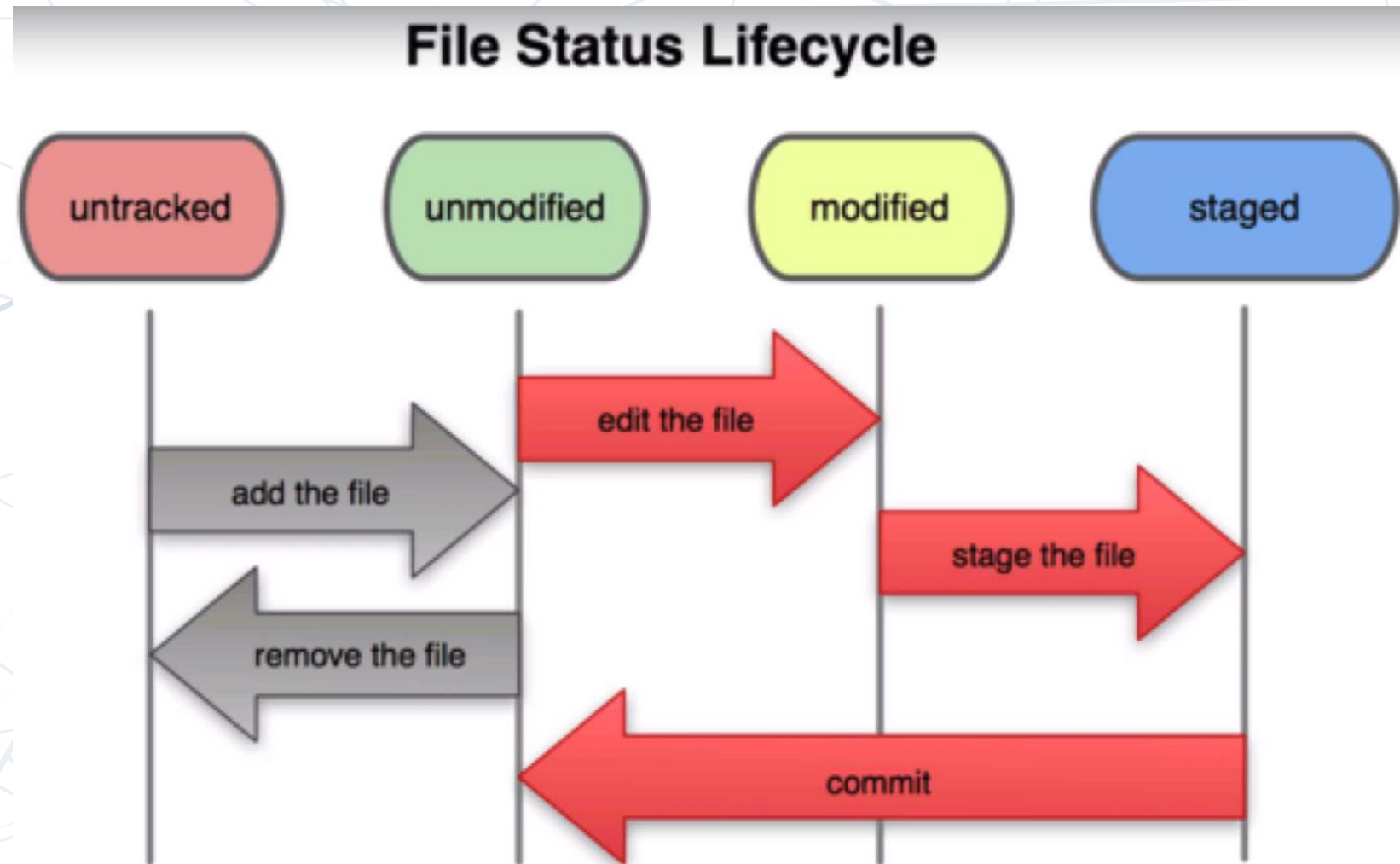
Arquivos neste estado estão na versão atual, foram alterados, mas não constarão na próxima versão.

Estado Staged

Arquivos neste estado trazem alguma mudança e constarão na nova versão.

Prontos para o Commit

Ciclo de vida de um arquivo



Para adicionar arquivos modificados (modified) ou não-rastreados (untracked) à staging area, existe o comando `git add`

Adicionando um arquivo:

```
git add arquivo1
```

Adicionando múltiplos arquivos:

```
git add arquivo2 arquivo3 arquivo4
```

Adicionando diretórios inteiros:

```
git add src/
```

É uma padrão comum adicionar todo diretório atual (lembre-se de que `.` significa "o diretório atual").

```
git add .
```

Há também a *flag* `-A` que adiciona todas as modificações, do projeto inteiro.

```
git add -A
```

Na prática, `git add .` e `git add -a` se comportam da mesma forma. Os dois só diferem quando o diretório atual não é a raiz do projeto, como por exemplo, o diretório `./src/`.

Git add

- git add .

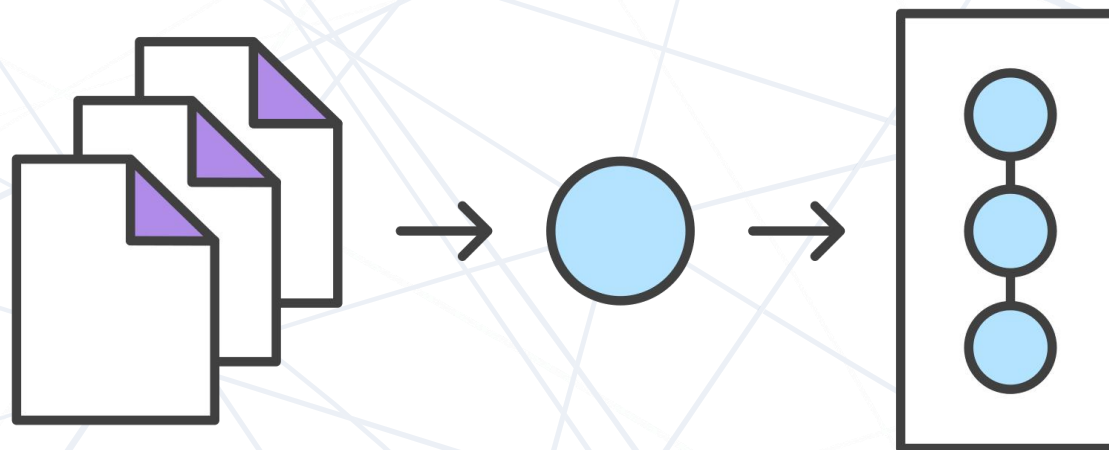
Verifica o status

```
tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME (master)
$ git add .

tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   css/styles.css
        new file:   index.js
```



Commit

Git commit

Nome significativo

Título com no máximo 50 letras

Verbos no indicativo

Mensagens com significado

Exemplo: Altera imagem de fundo

Alterar poucas coisas por commit

Git commit

- Somente as mudanças com estado "staged" farão parte do commit
- A todo commit, o Git associa um número chamado de hash. O número sempre é apresentado na sua forma hexadecimal, como por exemplo:
80ba4e49ee1fc346ee4934bdd2b4581e61b5c5fa.

Git commit

- `git commit -m 'Cria arquivo inicial'`

Verifica o status

```
tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME (master)
$ git commit -m "Cria arquivo inicial"
[master (root-commit) a4eebff] Cria arquivo inicial
2 files changed, 1 insertion(+)
create mode 100644 css/styles.css
create mode 100644 index.js

tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME (master)
$ git status
On branch master
nothing to commit, working tree clean
```

Padrões de commit

Convenções para escrever mensagens claras e estruturadas para o histórico de controle de versão, facilitando a leitura e colaboração

- Conventional Commits

<https://www.conventionalcommits.org/en/v1.0.0/>

Estrutura básica

<tipo>[(escopo opcional)]: <descrição>

[corpo da mensagem (opcional)]

[rodapé(s) (opcional)]

<tipo> (Obrigatório)

Uma palavra reservada que descreve a natureza da alteração.

- feat: Uma nova funcionalidade (feature).
- fix: A correção de um bug (problema).
- docs: Alterações na documentação.
- style: Mudanças de estilo, formatação (espaços, ponto e vírgula), que não afetam a lógica do código.

<tipo> (Obrigatório)

- refactor: Uma mudança de código que não corrige um bug nem adiciona uma funcionalidade (ex: renomeação de variáveis, reestruturação de código existente).
- test: Adição ou alteração de testes.
- chore: Mudanças na configuração, scripts de build, tarefas de automação, etc..

[(escopo opcional)] (Opcional)

Indica a parte específica do projeto que foi afetada pela mudança, geralmente entre parênteses. Ajuda a dar mais contexto

(ui): Alterações na interface do usuário ou nos componentes visuais.

(docs): Mudanças na documentação (arquivos Markdown, comentários de código, etc.).

(tests): Adição ou modificação de testes (unitários, de integração, e2e).

[(escopo opcional)] (Opcional)

(build): Alterações no sistema de build ou nas dependências externas (ex: webpack, babel, package.json).

(ci): Alterações nos arquivos e scripts de Integração Contínua (ex: Jenkins, GitHub Actions, TravisCI).

(core): Mudanças no núcleo ou na lógica principal da aplicação.

(shared): Modificações em códigos, módulos ou componentes compartilhados entre diferentes partes do projeto.

[(escopo opcional)] (Opcional)

(deps): Atualização ou adição de dependências.

(lint): Ajustes em regras de lint ou formatação de código.

(style): Alterações de estilo (CSS, SCSS) que não afetam a lógica ou o comportamento do código.

<descrição> (Obrigatório)

Uma descrição concisa e imperativa do que o commit faz.

Deve ser curta (preferencialmente até 50 caracteres) e escrita no tempo presente

Exemplo: "Adiciona autenticação"

Corpo e Rodapé (Opcionais)

Além do cabeçalho, a mensagem pode ter um corpo e um rodapé, separados por linhas em branco:

- Corpo: Fornece detalhes mais aprofundados sobre a mudança, como a motivação e o que ela resolve.
- Rodapé: Pode conter informações adicionais, como referências a problemas (Ref #123) ou a indicação de uma BREAKING CHANGE (mudança que quebra a compatibilidade).

Exemplos

Documentação:

- docs(readme): Atualiza a seção de instalação no README

Refatoração (sem mudança de comportamento):

- refactor(code): Refatora a lógica de validação de formulário

Testes:

- test(ci): Adiciona testes unitários para o novo serviço de cache

Exemplos

Estilo (formatação):

- `style(format)`: Formata o código com prettier

Funcionalidade com corpo explicativo:

- `feat(emails)`: Adiciona template de e-mail para redefinição de senha

Adiciona um novo template de e-mail usando Handlebars para a funcionalidade de recuperação de senha. Garante que o link gerado expire em 24 horas.

Exemplos

Correção de bug com referência a issue:

- `fix(db)`: Corrige erro de conexão com o banco de dados

O cliente do banco de dados estava usando credenciais antigas.

Atualiza para as credenciais corretas.

Refs: #123, #124

OBSERVAÇÕES GIT COMMIT

Para alterar a mensagem de commit ou adicionar arquivos:

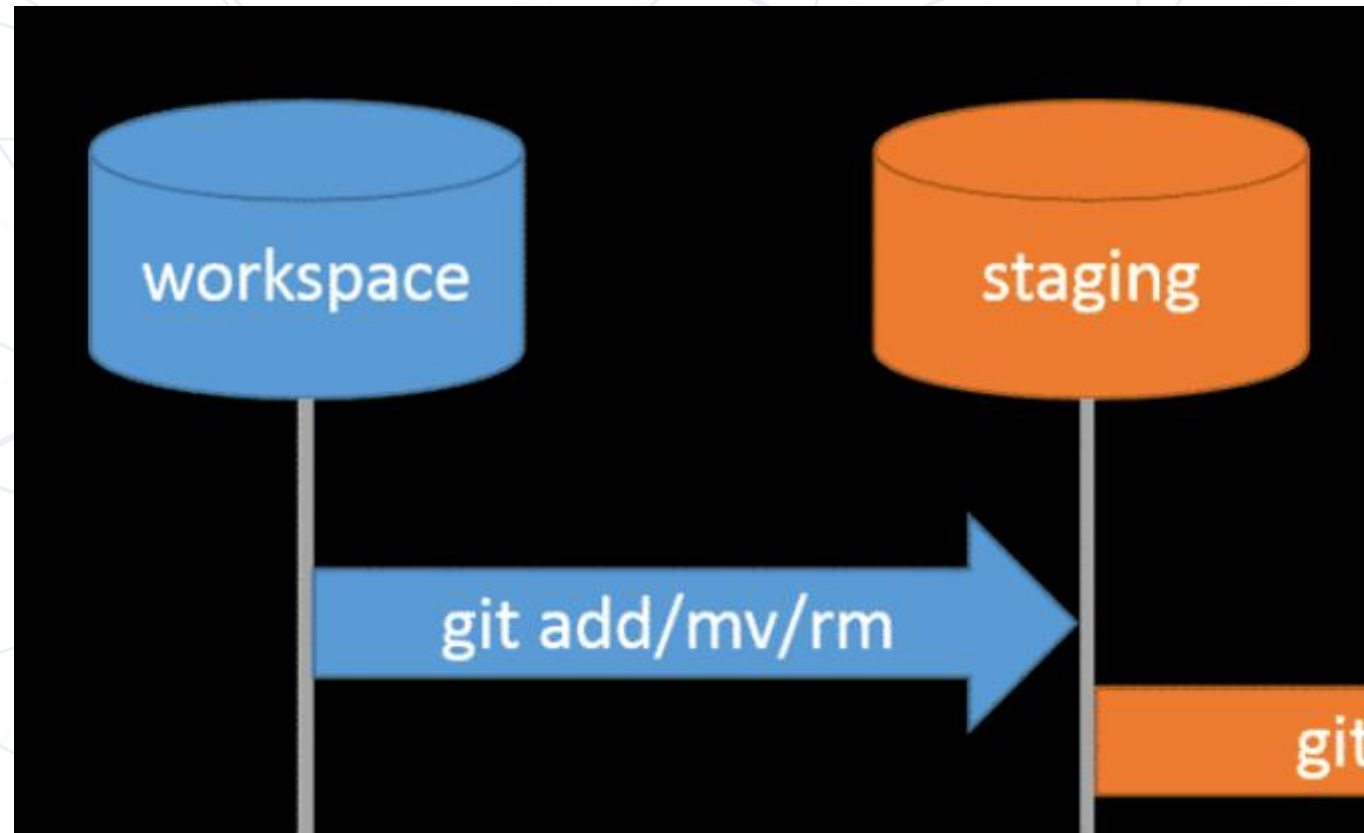
- `git commit --amend`

Para desfazer o commit completamente:

- `git reset --soft HEAD~1`

Este comando removerá o último commit do histórico, mas manterá todas as alterações no staging area e no diretório de trabalho, prontas para serem commitadas novamente.

Se precisar retroceder mais de um commit troque o 1 pela quantidade necessária.



Git rm - Git mv

Git rm

Função: Remove um arquivo do repositório Git e também do sistema de arquivos local (diretório de trabalho).

É o inverso do git add para novos arquivos.

Uso: `git rm <nome_do_arquivo>`

Observação: Se você quiser apenas parar de rastrear um arquivo, mas mantê-lo no sistema de arquivos, use `git rm --cached`.

Removam o arquivo index.js

Verifiquem o status

```
tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME/repositorio_portfolio_web (main)
$ git rm index.js
rm 'index.js'

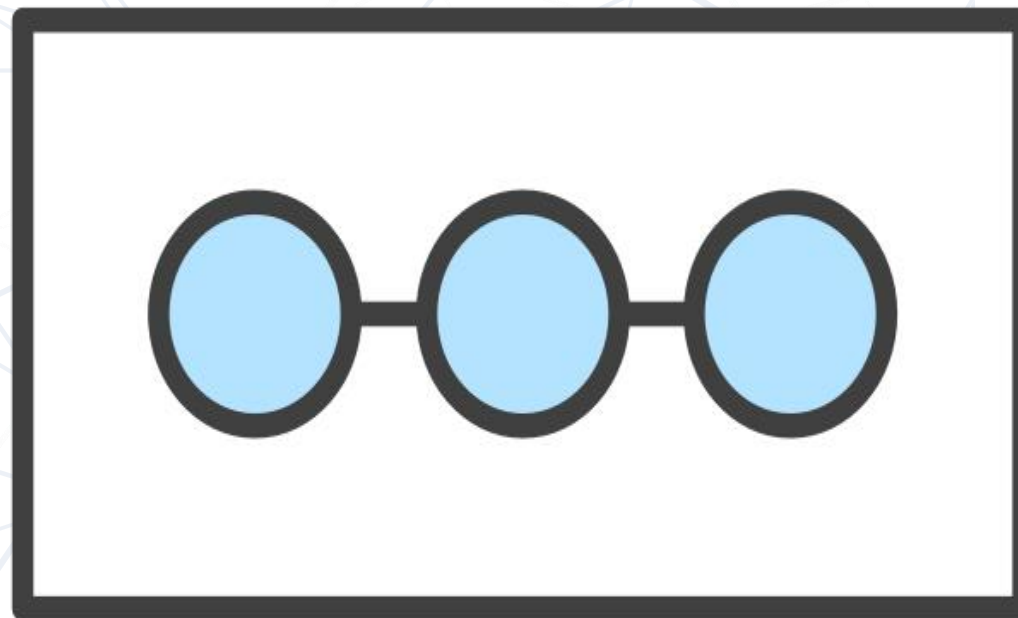
tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME/repositorio_portfolio_web (main)
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    index.js
```

git mv

Função: Move ou renomeia um arquivo ou diretório, garantindo que o Git registre essa operação como uma renomeação em vez de uma exclusão seguida por uma nova adição, o que preserva o histórico do arquivo.

Uso: `git mv <origem> <destino>`

Vantagem: Simplifica o processo e permite que o Git rastreie o histórico de renomeação de forma mais limpa. Embora seja possível fazer manualmente com `mv`, `git rm` e `git add`, o `git mv` automatiza e esclarece a intenção.



Visualizando Branches

Git log e Git show

Git log

O comando git log permite visualizar o histórico do repositório.

```
tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME (master)
$ git log
commit a4eebffc7016755d30929a7d80433f398501d5d2 (HEAD -> master)
Author: JeanMaxSKrebs <jeanmaxskrebs@gmail.com>
Date:   Sun Nov 2 18:14:44 2025 -0300
```

Cria arquivo inicial

Git log

É possível compactar a formatação do comando:

```
tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME (master)
$ git log --oneline
a4eebff (HEAD -> master) Cria arquivo inicial
```

Por padrão, o git log só exibe conteúdo da branch atual.

Git Show

- git show

```
tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME (master)
$ git show
commit a4eebffc7016755d30929a7d80433f398501d5d2 (HEAD -> master)
Author: JeanMaxSKrebs <jeanmaxskrebs@gmail.com>
Date: Sun Nov 2 18:14:44 2025 -0300

    Cria arquivo inicial

diff --git a/css/styles.css b/css/styles.css
new file mode 100644
index 0000000..e69de29
diff --git a/index.js b/index.js
new file mode 100644
index 0000000..6f16acb
--- /dev/null
+++ b/index.js
@@ -0,0 +1 @@
+teste
```


É possível visualizar arquivos de outras versões

- `git show versão:arquivo`

```
tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME (master)
$ git show a4eebff:index.js
teste
```

Atividade Git: Refazer tarefas de HTML Introdução e CSS Básico (1/2)

Você deverá revisitar HTML Introdução e CSS Básico, criando um repositório organizado e utilizando Git com Conventional Commits.

Deverá incluir nos commits, o tipo, o escopo opcional e descrição.

Nesse atividade, o escopo opcional deverá incluir a pasta alterada ou arquivo ou modificação na estrutura, caso for um novo arquivo criado.

Atividade Git: Sequência ideal de commits

Estrutura inicial do projeto

- feat(estrutura): cria a estrutura inicial do projeto

Adicionado arquivo README, arquivo .gitignore

- feat(projeto_1-index.html): cria o arquivo
- feat(index.html): adiciona link para tags.html
- feat(index.html): adiciona imagem para o link
- feat(index.html): adiciona link para receitas.html
- feat(index.html): adiciona imagem para o link
- feat(projeto_1-tags.html): cria o arquivo

Atividade Git: Sequência ideal de commits

Modifica o arquivo tags.html

- feat(tags.html): modifica o arquivo
Inserido âncoras para os arquivos de resumo

Adiciona os arquivos da pasta imagens

- feat(imagens): adiciona imagem 1
descrição da imagem
- feat(imagens): adiciona imagem 2
descrição da imagem

Atividade Git: Sequência ideal de commits

Criar a estrutura da pasta resumo-tags

- feat(resumo_tags-estrutura.html): cria o arquivo
- feat(resumo_tags-citacoes.html): cria o arquivo

... mais 5.

Modificar os arquivos da pasta resumo-tags

- feat(estrutura.html): adiciona conteúdo

Adicionada a estrutura base do arquivo HTML e conteúdos introdutórios sobre a estrutura fundamental de documentos HTML

... mais 6.

Atividade Git: Sequência ideal de commits

Criar a estrutura da página de receita

- feat(receitas-listas_culinarias.html): cria o arquivo
- feat(receitas-imagens): adiciona imagem 1
descrição da imagem

mais 2 imagens...

Criar cada arquivo de receita separadamente (OBS: Mudar Genérico)

- feat(receitas-receita_1.html): cria o arquivo
- feat(receitas-receita_2.html): cria o arquivo
- feat(receitas-receita_3.html): cria o arquivo

Atividade Git: Sequência ideal de commits

Adicionar as receitas na página principal (index.html)

- feat(listas_culinarias.html): adiciona uma lista com imagens

Adicionada uma lista de links, com imagens, para receitas x,y e z

Criar listas e estrutura interna dentro de cada receita (OBS: Mudar Genérico)

- feat(receitas): adiciona lista à receita 1

Adicionada lista (não ordenada) de ingredientes da receita 1

- feat(receitas): adiciona lista (ordenada) de preparo à receita 1
- feat(receitas): adiciona lista (de definição), glossário à receita 1
- feat(receitas): adiciona lista (não ordenada) de ingredientes à receita 2

... outras 5 feat.

Atividade Git: Refazer tarefas de HTML Introdução e CSS Básico

Faça no mínimo mais 25 commits para os tópicos a seguir:

Manter padrões de tipo, o escopo opcional e descrição.

Inserindo as estilizações feitas com CSS (15 a 20 commits)

Criação da pasta css e arquivo style

Cada commit deve exemplificar o que o css altera

Exemplo: style(css ou receitas-css): Altera a cor de fundo

Inserindo o arquivo resultado.html e alterações Javascript (5 commits)

Criação do arquivo vazio

Primeira modificação do arquivo (Arquivo Base GitHub), constar no corpo da descrição do commit

Alterações feitas (3 commits) Exemplo: feat(arquivo e js): Altera bolo de cenoura → receita 1

Atividade Git: Refazer tarefas de HTML Introdução e CSS Básico

Faça no mínimo mais 30 commits para os tópicos a seguir:

Modificando o arquivo README (3 commits)

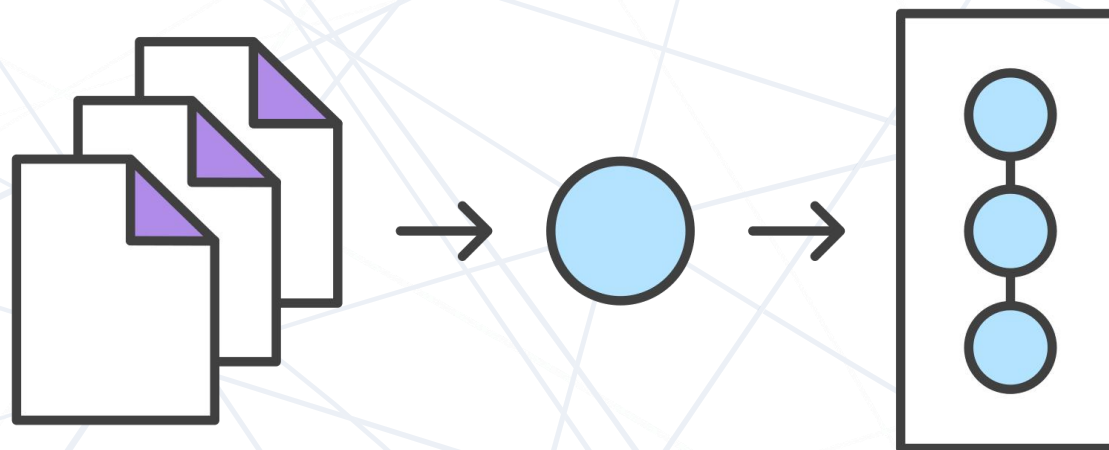
Exemplo: docs(README): adiciona introdução ao projeto

Movendo o arquivo tags.html (1 commit)

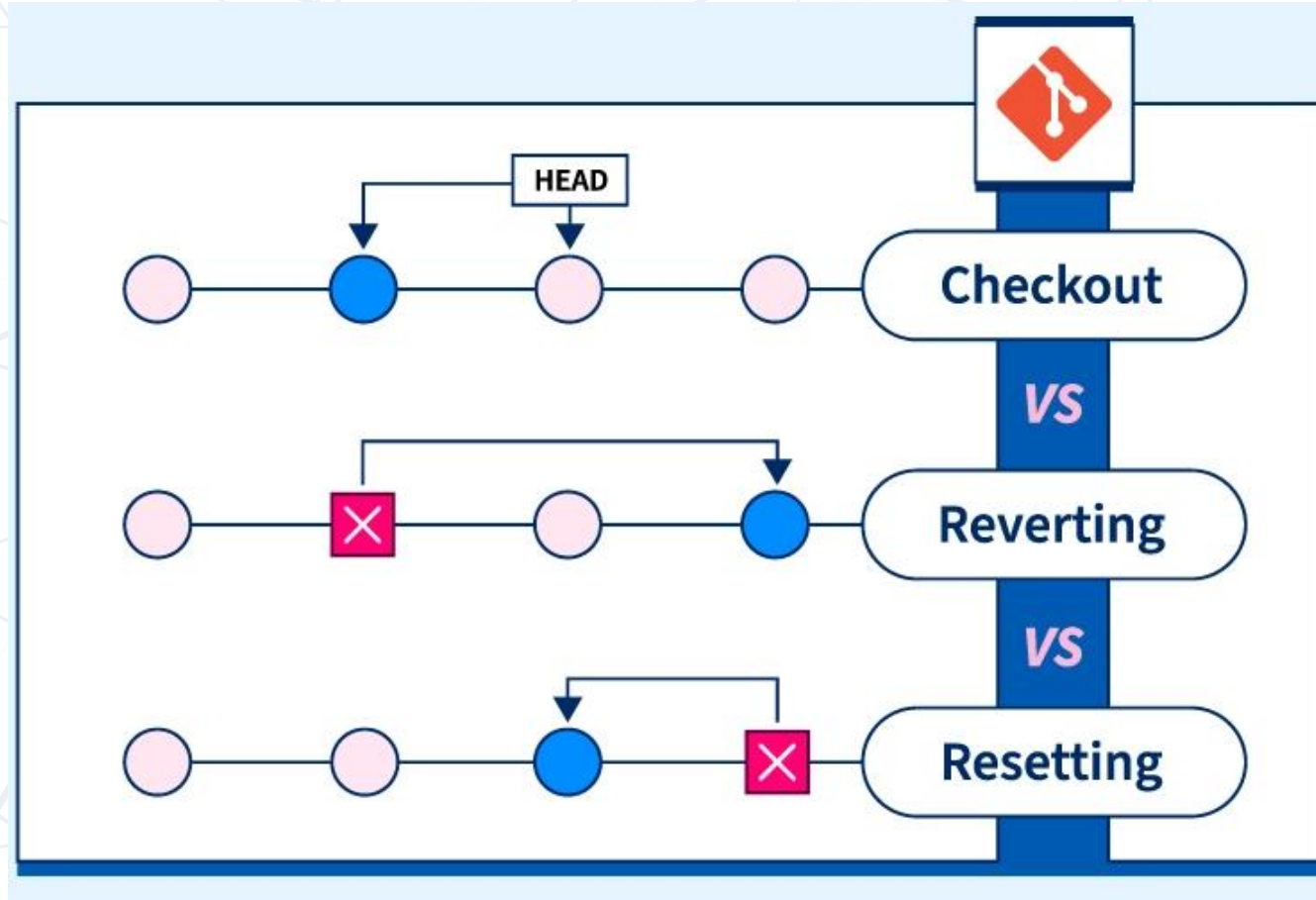
chore(estrutura): move tags.html para nova pasta

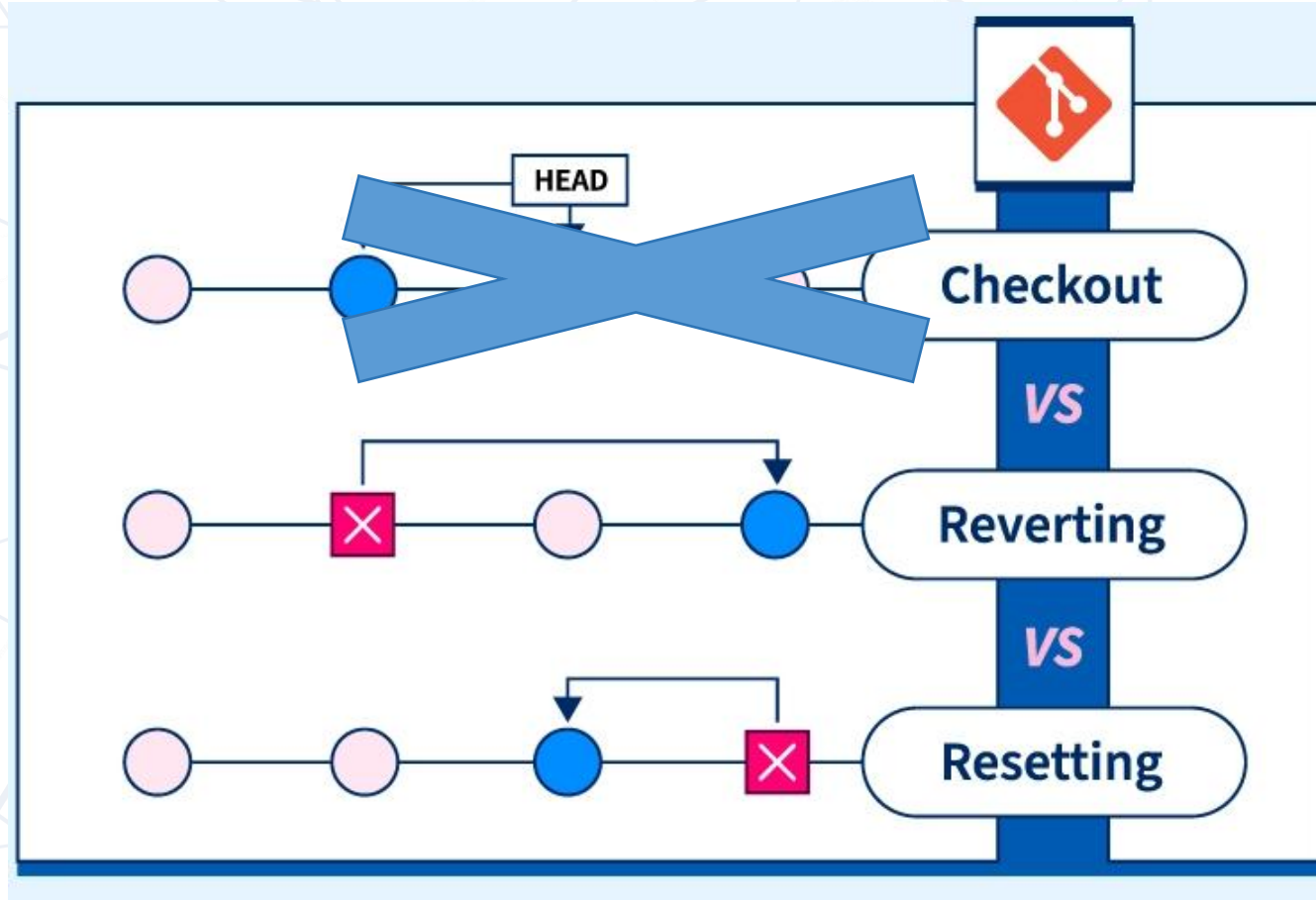
Fazendo as alterações nos outros arquivos sobre tags.html (1 commit)

fix(links): atualiza referências após mover tags.html



Gerenciar mudanças e histórico

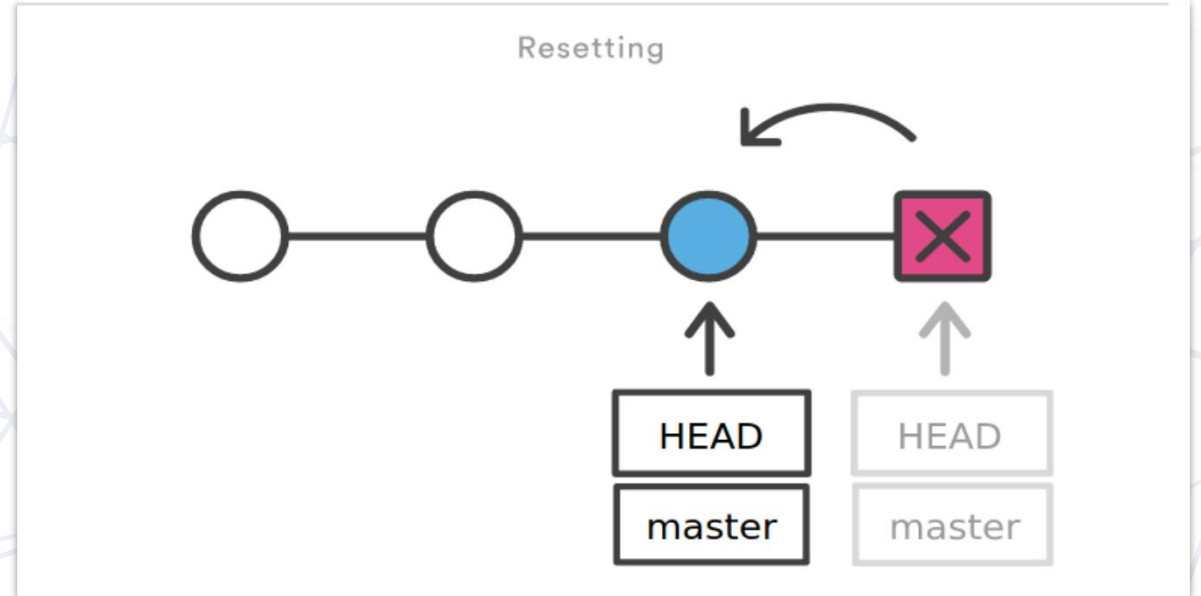




Git reset

Desfazer commits e reescrever o histórico do projeto

Uso principal: Desfazer commits locais (que ainda não foram compartilhados em um repositório remoto).



Git reset

- git reset

Verifique o status

Esse comando remove os arquivos da área de staging, mas mantém as alterações no diretório de trabalho.

```
tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME/repositorio_portfolio_web (main)
$ git reset
Unstaged changes after reset:
M    css/styles.css
D    index.js

tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME/repositorio_portfolio_web (main)
$ git status
On branch main
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   css/styles.css
        deleted:    index.js

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        LICENSE
        README.md
        css/reset.css
        css/themes/
        favicon.ico
        index.html

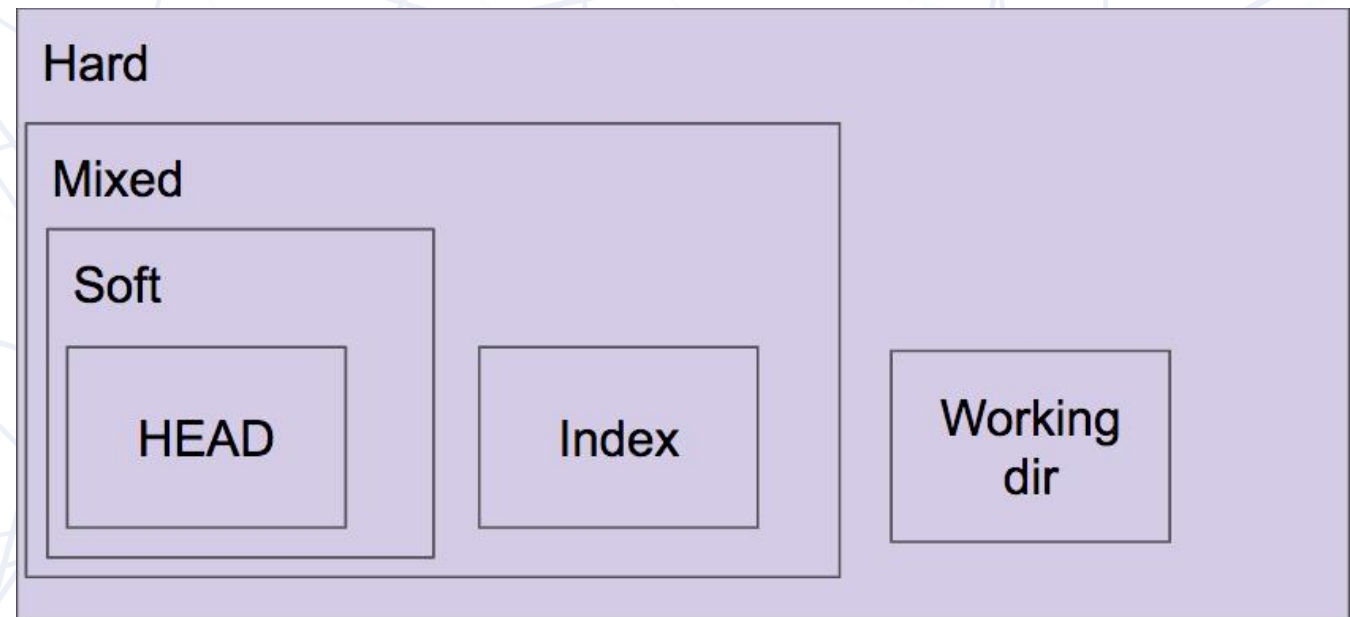
no changes added to commit (use "git add" and/or "git commit -a")
```

Modos de Operação:

--soft: Mantém as mudanças no índice e no diretório.

--mixed (padrão): Remove mudanças do índice, mas mantém no diretório.

--hard: Remove as mudanças do índice e do diretório.



git reset --soft

O commit voltaria, mas as mudanças continuariam staged.

index.js ainda apareceria como deletado e pronto para commit.

styles.css continuaria modificado e não staged.

Nada seria perdido.

Só mexe no histórico de commits. Área de staging e arquivos ficam intocados.

git reset --mixed

O commit volta.

index.js sai do staged → vira deleted (não staged).

styles.css continua modificado.

Nada no disco é apagado.

Desfaz o staging, mas não altera seus arquivos.

git reset --hard

O commit volta e todas as alterações seriam descartadas!

O index.js seria restaurado como estava no commit anterior (não mais deletado).

As modificações de styles.css seriam apagadas totalmente.

Arquivos não rastreados permaneceriam no diretório, mas não seriam adicionados ao Git.

Apenas arquivos rastreados voltam para o estado do commit — os arquivos não rastreados continuam no disco.

Apaga tudo o que não foi commitado. Perigosíssimo se não tiver backup!

Resumo

Situação após reset	soft	mixed	hard
<code>index.js</code> deletado, staged?	✓ Sim	✗ Não	✗ Não (e ainda restaura o arquivo)
Alterações no <code>styles.css</code>	✓ Mantém	✓ Mantém	✗ Apaga
Arquivos untracked	✓ Mantém	✓ Mantém	✓ Mantém
Risco de perda de trabalho	⚠ Baixo	⚠ Médio	✗ Alto

Git restore

Introduzido para tornar a ação de desfazer alterações no diretório de trabalho e na área de staging mais clara e separada do git reset.

Uso principal: Descartar alterações locais em arquivos.

Casos de Uso:

Descartar alterações no diretório de trabalho:

Remover arquivos da área de staging (unstage):

Git restore

Descartar alterações no diretório de trabalho

- `git restore <arquivo>`

Isso descarta quaisquer alterações locais não preparadas (unstaged).

Parecido com `git reset --hard`, exceto que arquivos novos não serão deletados.

Remover arquivos da área de staging (unstage):

- `git restore --staged <arquivo>`

Exatamente como `git reset --mixed`.

Git revert

- `git revert <hash_do_commit>`

Casos de Uso:

Commit 15: adicionou botão

Commit 16: arrumou estilo

Commit 17: quebrou layout

- `git revert HEAD`

Commit 18: “Revert ‘quebrou layout’”

O commit que causou o erro continua no histórico

Git reset --hard HEAD~n X Git Revert

- `reset --hard HEAD~n`

É destrutivo. Apaga dados. Reescreve o histórico.

Apaga o commit mais recente.

Apaga todas as alterações no diretório de trabalho.

Não deve ser usado depois que o commit já foi enviado ao GitHub, pois quebra o histórico da equipe.

- `git revert HEAD`

Não apaga nada. Apenas adiciona um commit que desfaz outro.

Cria um novo commit que desfaz (inverte) as alterações do último commit.

Nada é apagado.

É seguro para usar mesmo depois de enviar ao GitHub.

Exercícios

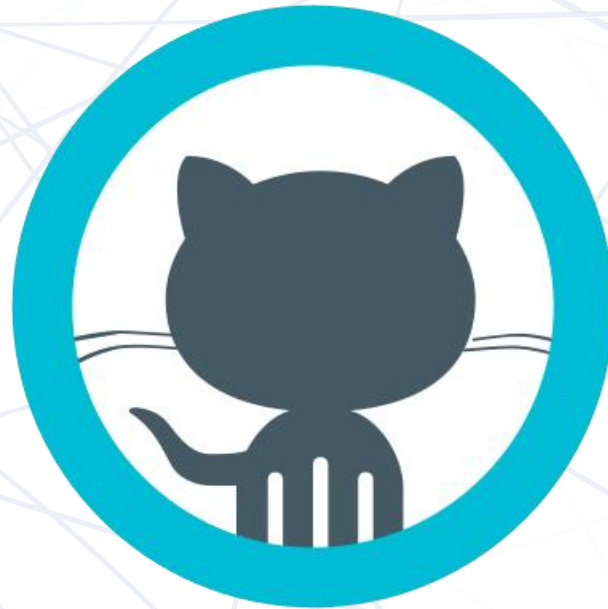
- Você modificou apenas um arquivo, mas ainda não deu git add. Agora quer descartar completamente essas alterações.
- Você já usou git add em um arquivo por engano e quer removê-lo da área de staging, mas sem perder as alterações feitas nele.
- Você fez alterações em vários arquivos e quer remover todos eles do staging de uma vez, sem perder nada no diretório de trabalho.
- Você quer voltar um commit para trás, mas manter todas as alterações ainda no diretório de trabalho.

Exercícios

- Você modificou apenas um arquivo, mas ainda não deu git add. Agora quer descartar completamente essas alterações. `git restore <arquivo>`
- Você já usou git add em um arquivo por engano e quer removê-lo da área de staging, mas sem perder as alterações feitas nele. `git restore --staged <arquivo>`
- Você fez alterações em vários arquivos e quer remover todos eles do staging de uma vez, sem perder nada no diretório de trabalho. `git restore --staged .`
- Você quer voltar um commit para trás, mas manter todas as alterações ainda no diretório de trabalho. `git reset --soft HEAD~1`



Etapa 3 - Repositório Remoto



Conectando com o Github

Github

- Criar repositório(Github)
- seuUsuario.github.io
- Adicionar Descrição

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1

General

Owner *

ProfessorJeanMax

Repository name *

professorjeanmax.github.io

⚠ professorjeanmax.github.io already exists in this account

Great repository names are short and memorable. How about **automatic-octo-robot**?

Description

Portfólio Pessoal

17 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

Github

Configuração rápida para o GitHub Pages

No nome do repositório, clique em Configurações (Settings). Caso não consiga ver a guia Configurações, selecione o menu suspenso , clique em Configurações.

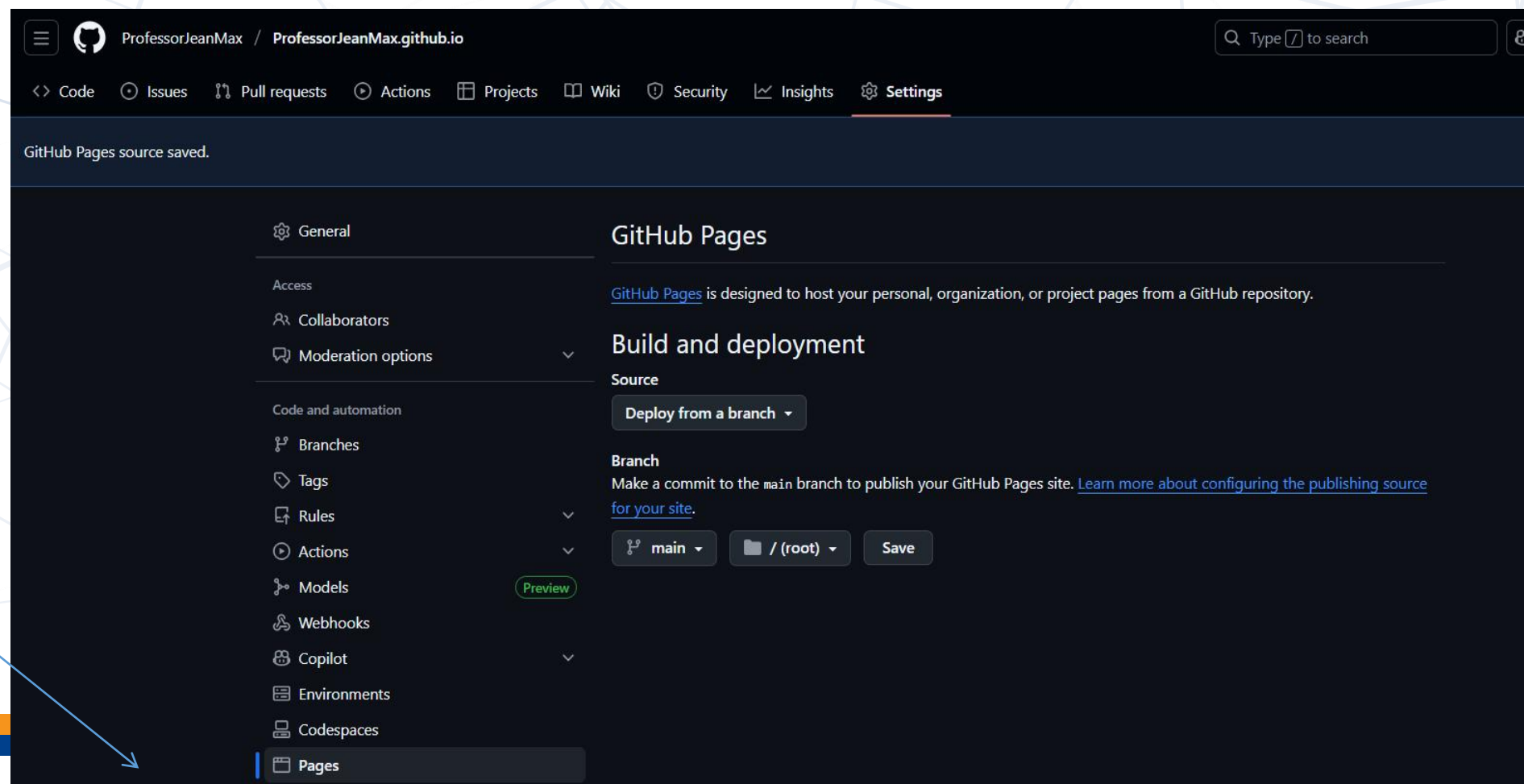
Na seção "Code and automation" da barra lateral, clique em Pages.

Em "Build e implantação", em "Fonte", selecione Fazer implantação de um branch.

Em "Build e implantação", em "Branch", use o menu suspenso do branch e selecione uma fonte de publicação.

Github

Configuração rápida para o GitHub Pages



Repositório Remoto

O repositório que você tem salvo na sua máquina local é um repositório independente.

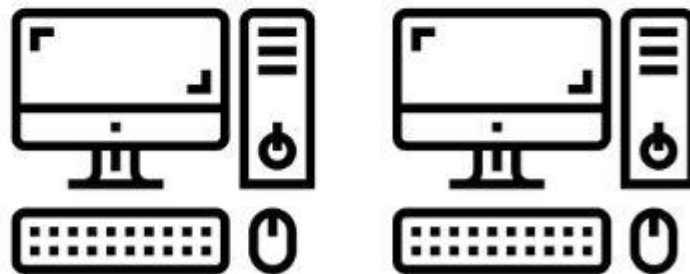
O repositório local é uma cópia do código-fonte que fica no seu computador e onde você faz as alterações e commits.

O repositório remoto é uma cópia hospedada em um servidor (como GitHub) usada para colaboração e backup.

Repositório local



Repositório Remoto



Acesso à colaboradores

Repositório Remoto

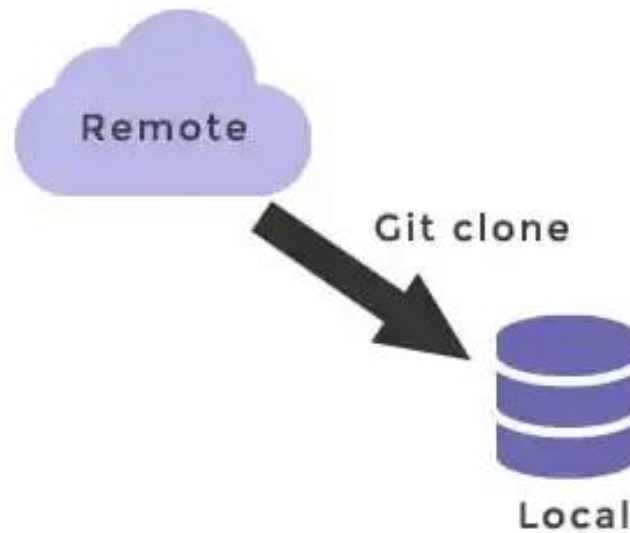
Alternativas



Repositório Azure Git



GitLab



Clonando Repositórios Remotos

Git Clone

A porta de entrada para o ecossistema de controle de versão distribuído do Git, permitindo que equipes trabalhem de forma organizada, transparente e eficiente em projetos de código.

Ele cria uma cópia completa de um repositório existente em um novo diretório na sua máquina local.

Template

Hello, I'm Dinesh!

Full Stack Dev | Open-Source Enthusiast | Blogger | Ninja



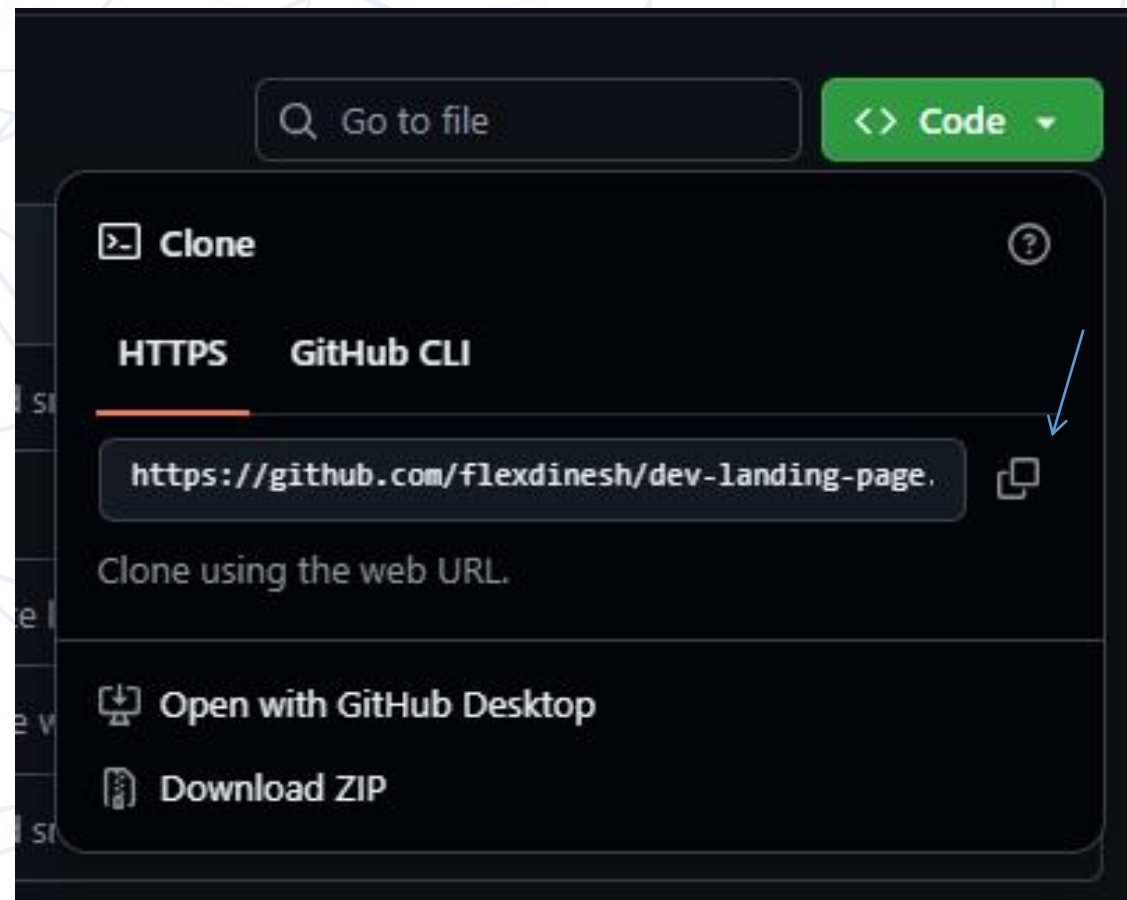
<https://git.io/JegaO>

Git Clone

Entrar no repositório
fornecido da página anterior

Procurar o botão verde
Code/Código

Copiar a URL



Git Clone

No Git Bash, volte a pasta
NOME_SOBRENOME

- `cd ../`
- `git clone URL COPIADA template_portfolio`

```
tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME
$ git clone https://github.com/flexdinesh/dev-landing-page.git template_portfolio
Cloning into 'template_portfolio'...
remote: Enumerating objects: 69, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 69 (delta 1), reused 0 (delta 0), pack-reused 64 (from 2)
Receiving objects: 100% (69/69), 15.27 KiB | 1.17 MiB/s, done.
Resolving deltas: 100% (30/30), done.
```

Git Clone

Entre na pasta repositorio_portfolio_web

- `cd ./repositorio_portfolio_web`

Mover arquivos do template para o repositório e excluir o repositório antigo:

- `cp -r ../template_portfolio/* .`
- `rm -rf ../template_portfolio`

Git Clone

Verifica o status

```
tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME/repositorio_portfolio_web (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   css/styles.css

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        LICENSE
        README.md
        css/reset.css
        css/themes/
        favicon.ico
        index.html

no changes added to commit (use "git add" and/or "git commit -a")
```

Atividade Conjunta

Alterar Campos <link rel="stylesheet" >

Para comentar e descomentar “CTRL” + “;”

Atividade Conjunta

Fazer o commit do template com a descrição correta

Exemplo: Altera a cor de fundo para branco anil

Git remote

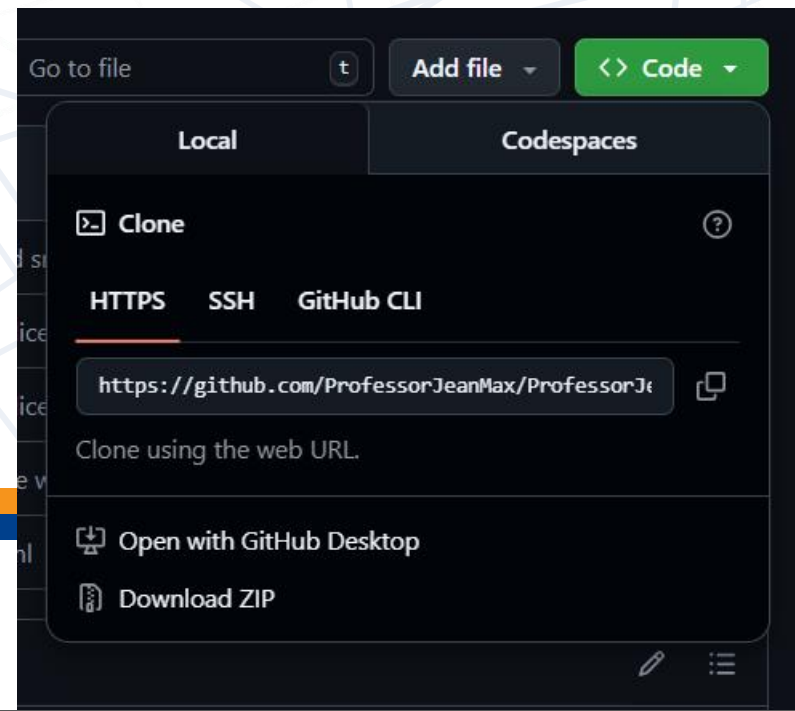
Por padrão, quando você usa git clone, ele salva um remoto chamado origin.

- git remote

```
tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME/repositorio_portfolio_web (main)  
$
```

Para adicionar um remoto, com nome origin

- git remote add origin URL



Git remote

Depois de adicionar verifique:

- git remote

```
tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME/repositorio_portfolio_web (main)
$ git remote add origin https://github.com/ProfessorJeanMax/ProfessorJeanMax.github.io.git

tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME/repositorio_portfolio_web (main)
$ git remote
origin
```

Para redefinir

- git remote set-url origin URL

Para remover

- git remote remove origin

Git push

Para mandar suas mudanças a um remoto salvo como origin numa branch main, use:

- `git push origin main`

Você pode omitir a branch se na primeira vez você utilizar a flag `--set-upstream` ou ``-u'`:

- `git push origin --set-upstream main`

ou

Git push

```
git push origin -u main
```

Nos pushes subsequentes, basta executar:

- `git push`

Nota: O GitHub usa main como a branch padrão, não master, por isso você precisa renomear a master para main antes de enviar ao GitHub.

Git pull

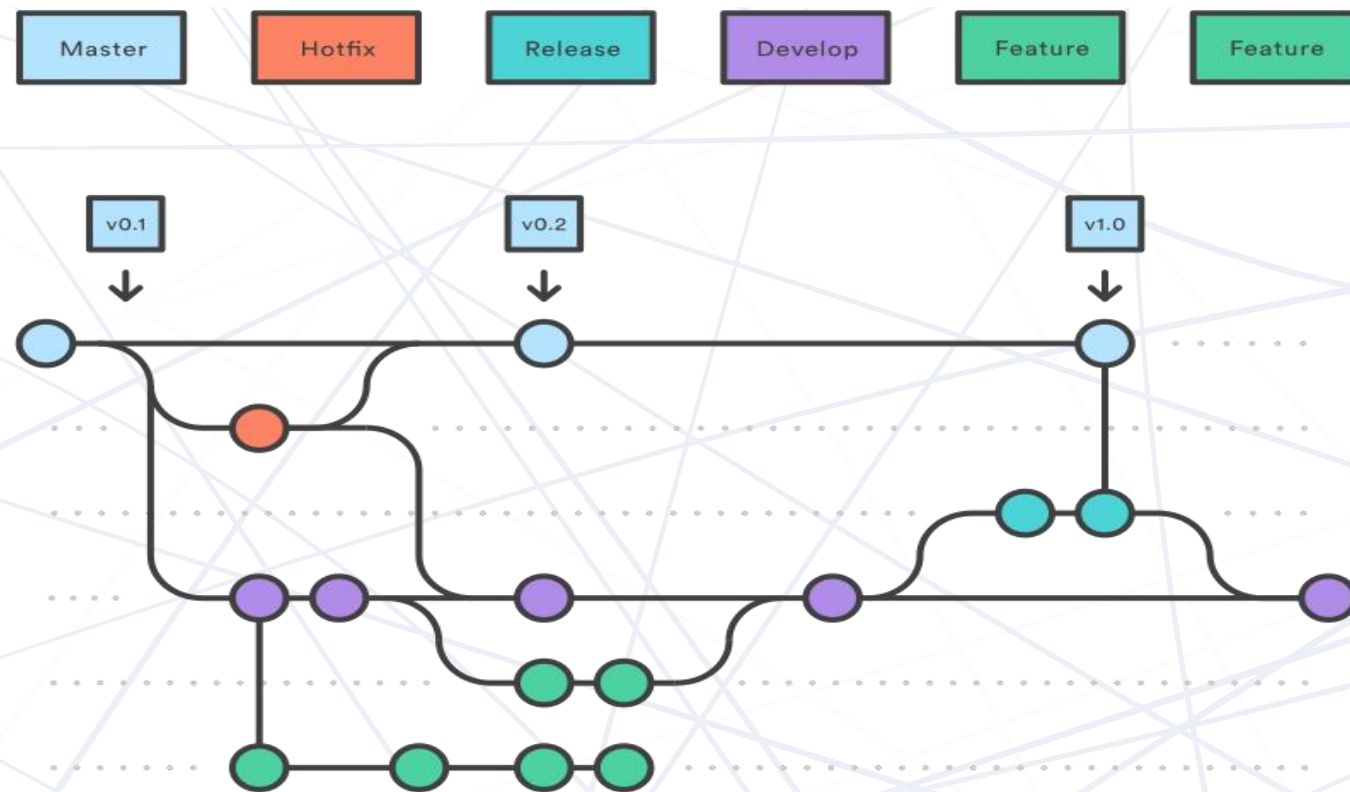
Semelhantemente, para receber mudanças de um repositório remoto salvo como origin numa branch main, use:

- `git pull origin main`

Você pode omitir a branch se na primeira vez você utilizar a flag `--set-upstream` :

- `git pull origin --set-upstream main`

Nota: Se repositório local tiver novos commits, e o remoto também tiver novos commits, divergindo a respeito da história da branch, você terá de resolver um merge.



Etapa 7 - Git branch e Git Merge

Git branch

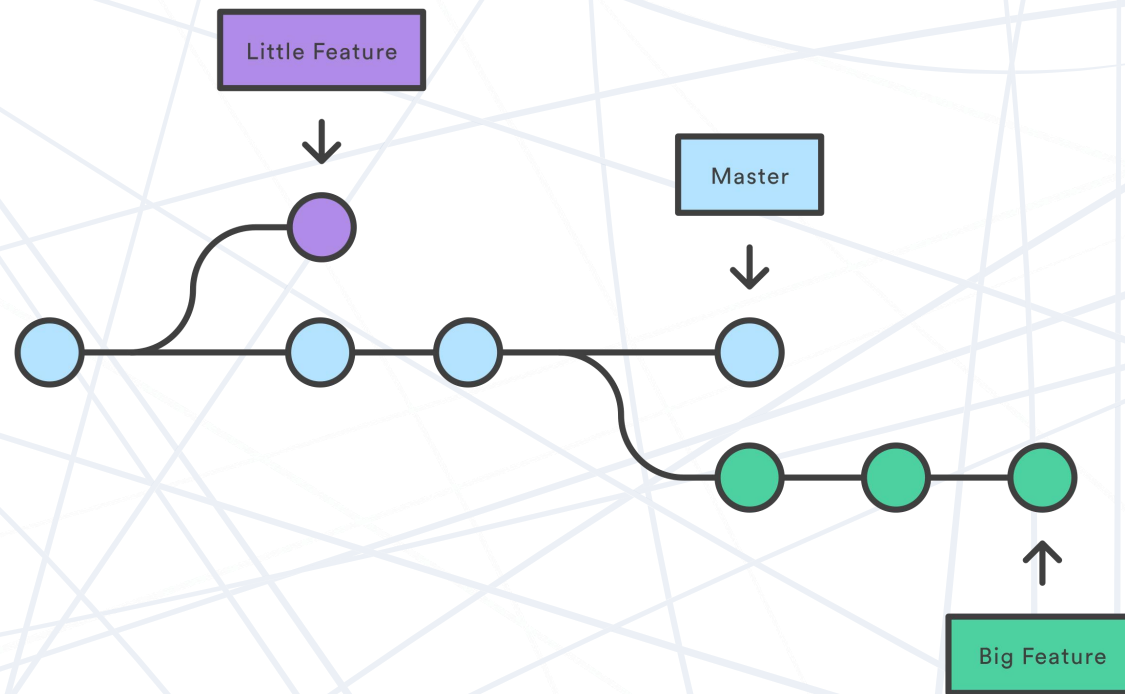
No Git, os commits são arranjados em uma linha do tempo. Por padrão, essa linha do tempo é linear, mas o Git também permite expressar bifurcações na linha do tempo. Para isso, existe o conceito de "branch" (literalmente "galho", "ramo").

A primeira branch criada é main para novos repositórios, enquanto a antiga padrão era master. A partir de outubro de 2020, o GitHub e o Git passaram a usar "main" como a branch padrão para novos repositórios, em substituição a "master".

Branch Master (Main)

a versão atual do seu
sistema

E as outras branches são
funcionalidades não
finalizadas



Renomeando Branch

Renomear sua master para main com o seguinte comando:

- `git branch -m master main`

```
tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME/repositorio_portfolio_web (master)
$ git branch -m master main

tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME/repositorio_portfolio_web (main)
$
```

Renomeando Branch

Se a branch alvo for a branch atual, o nome dela pode ser omitida

- `git branch -m main`

Branch

Uma branch é uma ramificação na linha do tempo do Git.

Criando Branch

```
git branch NOMEdaBRANCH
```

Simplesmente cria uma branch e não faz mais nada.

Trocando Branch

Por padrão, você não vai estar na branch que criou.

Para trocar de branch use:

- `git checkout NOMEdaBRANCH`

```
tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME/repositorio_portfolio_web (master)
$ git branch -m master main

tsije@DESKTOP-1KRUSLS MINGW64 ~/Desktop/NOME_SOBRENOME/repositorio_portfolio_web (main)
$
```


Criando e trocando de Branch

Para criar e trocar de branch ao mesmo tempo

- `git checkout -b NOMEdaBRANCH`

Criando branch por commit

Por padrão, a nova branch começa do commit atual (HEAD).

É possível criar uma branch a partir de um commit.

Lembre do que foi visto em GIT LOG e GIT SHOW

- `git branch NOMEdaBRANCH commitEscolhido`

Exemplo

Suponhamos um *commit* `3c97506` seguido de um *commit* `ec49a96`, ambos dentro da branch `master`. A `HEAD` aponta para este último. Temos uma árvore assim:

```
master
ec49a96 <- HEAD
|
3c97506
```



Após fazermos uma nova *branch* `teste` (`git checkout -b teste`), e produzirmos um novo *commit* `4b12d0a` nela, temos a seguinte:

```
      teste
      4b12d0a <- HEAD
     /
master /
ec49a96
|
3c97506
```



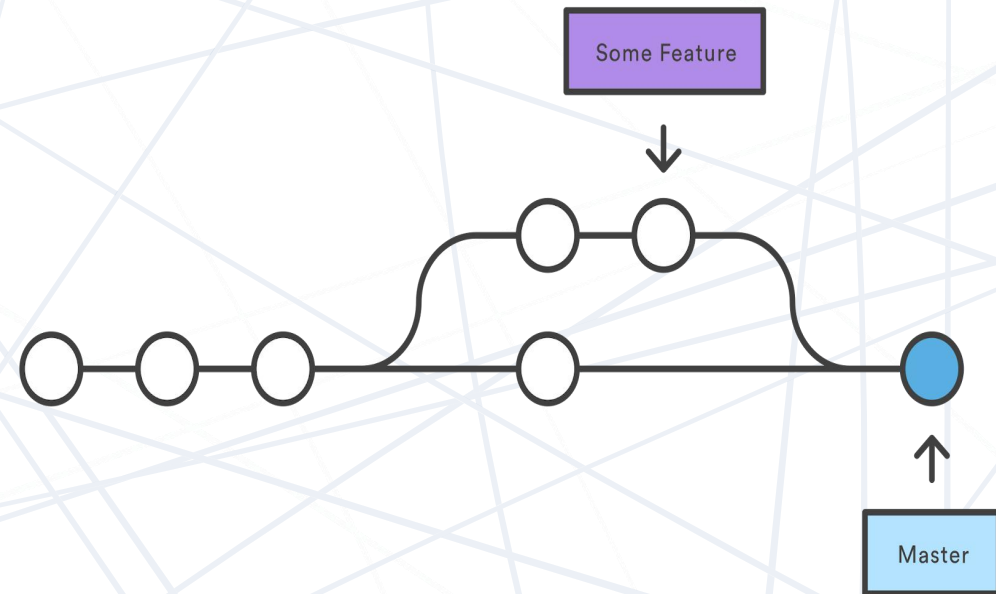
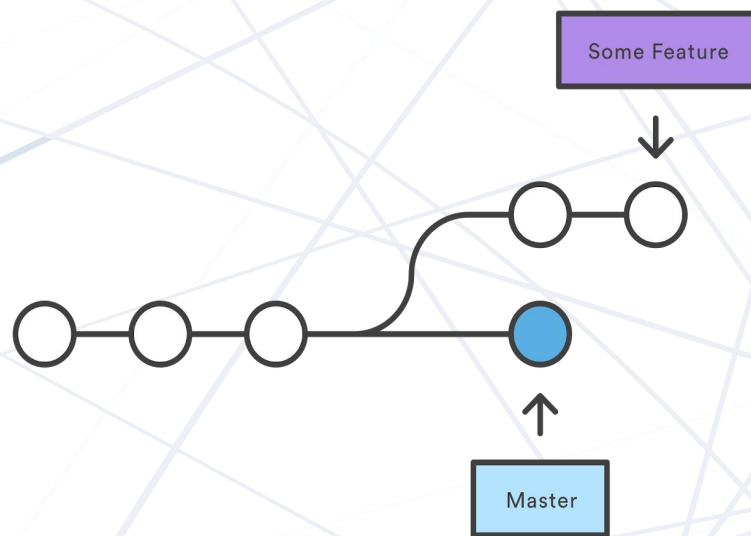
Se voltarmos para a `master` (`git checkout master`), e produzirmos um novo *commit* `80ba4e4`, temos:

```
      master  teste
      80ba4e4 4b12d0a
      |      /
      |     /
ec49a96
|
3c97506
HEAD ->
```



Branch

Fluxo de trabalho



Deletando Branch

Para deletar (requer que você tenha feito merge)

- `git branch -d NOMEdaBRANCH`

Para forçar

- `git branch -D NOMEdaBRANCH`

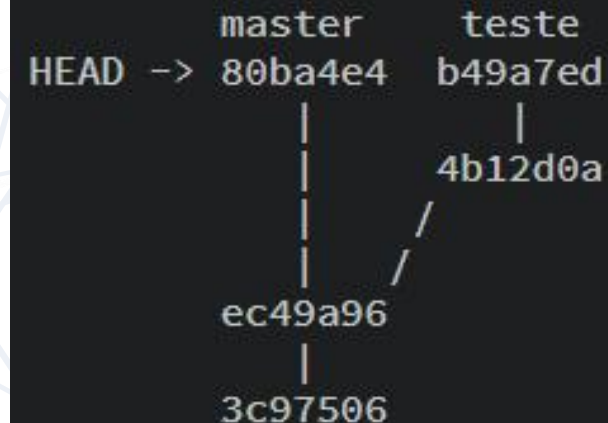
Git Merge

Esses diferentes "galhos" na linha do tempo, podem eventualmente juntar-se novamente, no que chamamos de merge (literalmente "união" ou "mescla")

Por exemplo, suponha que estejamos em uma branch "master" e queremos mesclar com outra branch "teste"

Suponha que queiramos aplicar as os dois commits `4b12d0a` e `b49a7ed` à branch master

- `git merge teste`

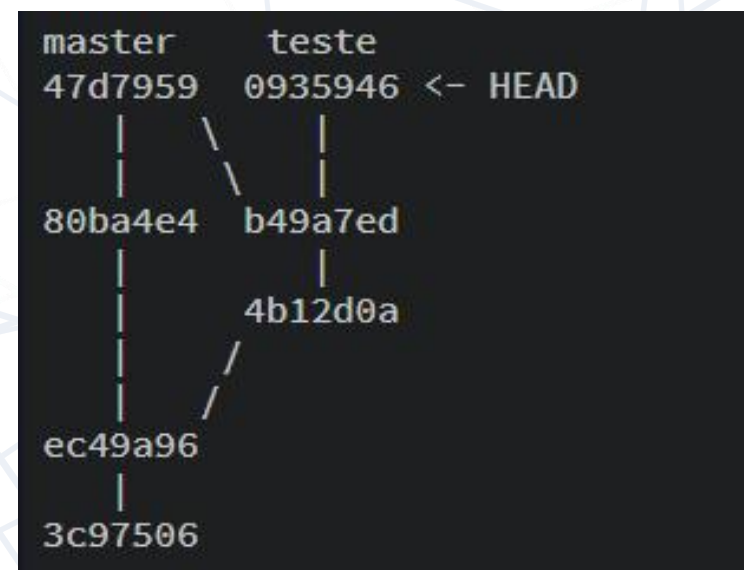


Git Merge

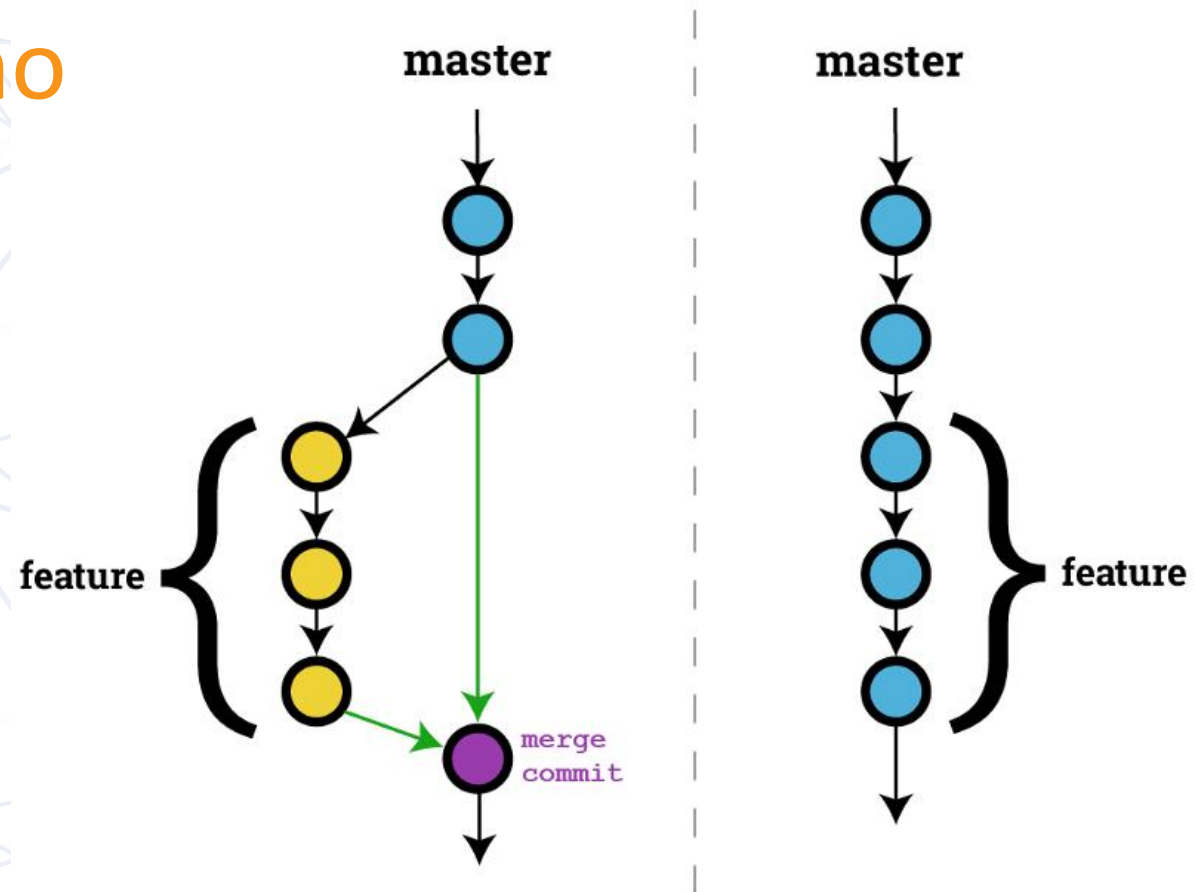
O resultado será



A branch teste continuará existindo.
Podemos fazer um novo commit nela



Fluxo de trabalho



Múltiplas Branches

Por que essa estratégia é adotada?

- Trabalho Paralelo
- Isolamento e Segurança
- Revisão de Código
- Histórico Organizado
- Facilidade de Rollback

Múltiplas Branches

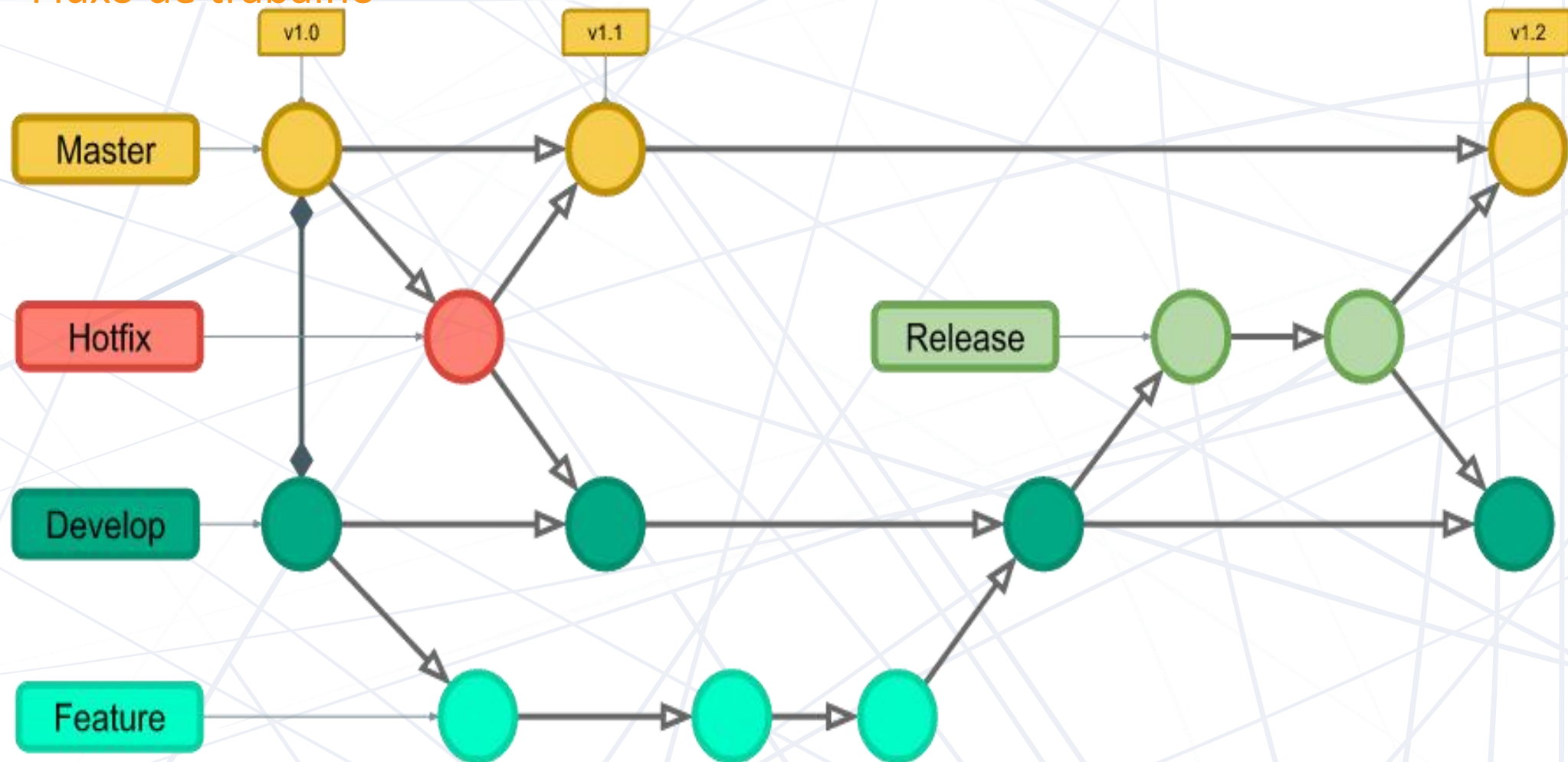
Fluxos de Trabalho Comuns

GitFlow: Um modelo mais complexo e robusto, ideal para projetos com lançamentos agendados e que exigem um ciclo de vida de suporte mais longo.

- main: Contém o código de produção, a sempre estável.
- develop: Contém o histórico completo do desenvolvimento e é a base para novas funcionalidades.
- feature: Para desenvolvimento de novas funcionalidades, criadas a partir de develop.
- release: Para preparar um novo lançamento, permitindo pequenos ajustes e testes finais antes da fusão com main.
- hotfix: Para correções urgentes em produção, criadas a partir de main e mescladas de volta em main e develop.

Git Flow

Fluxo de trabalho



Múltiplas Branches

Fluxos de Trabalho Comuns

GitHub Flow: O GitHub Flow é uma metodologia prática e direta que se adapta bem a projetos que necessitam de um ciclo de lançamento rápido e contínuo, em um processo direto e cíclico:

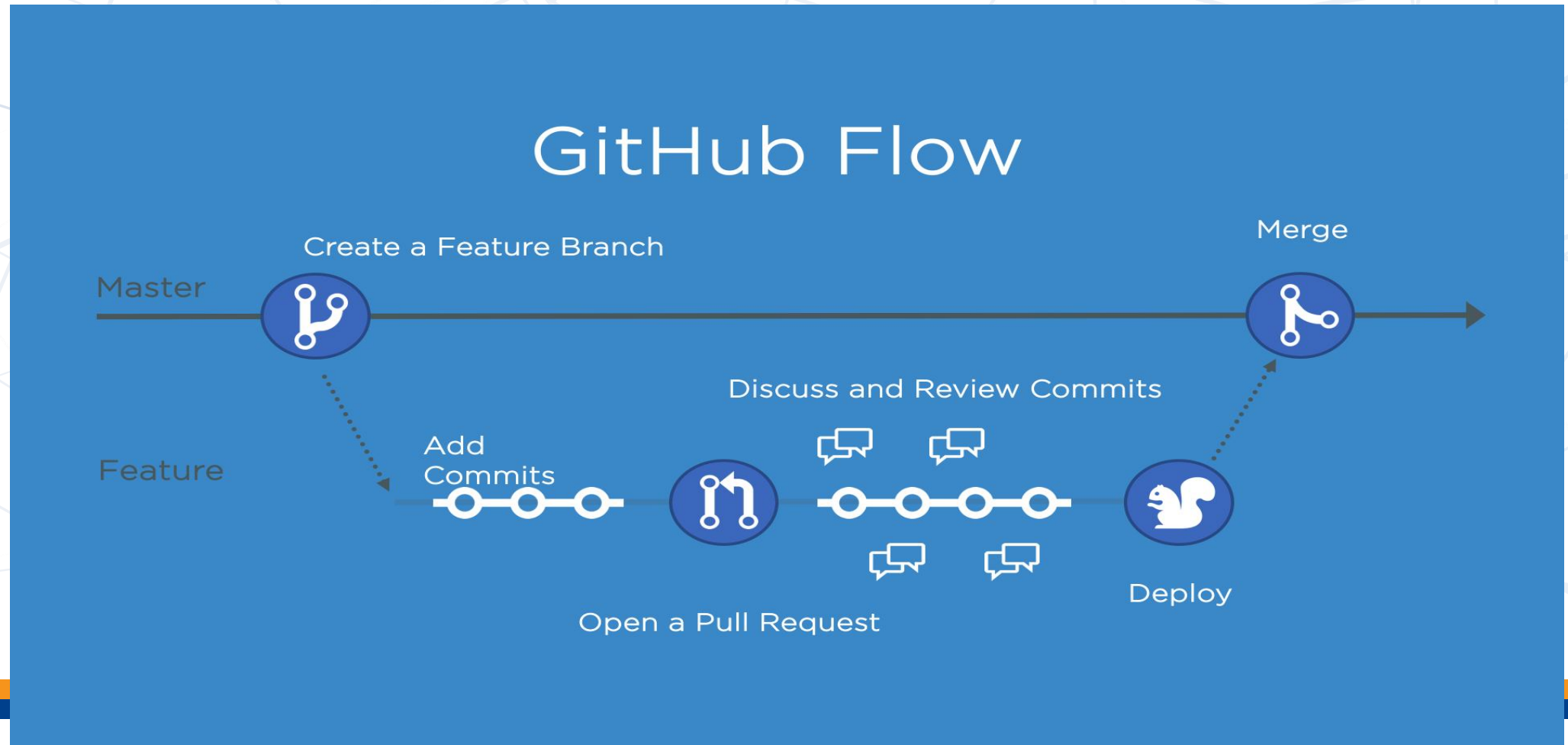
A branch main é sempre o ambiente de produção

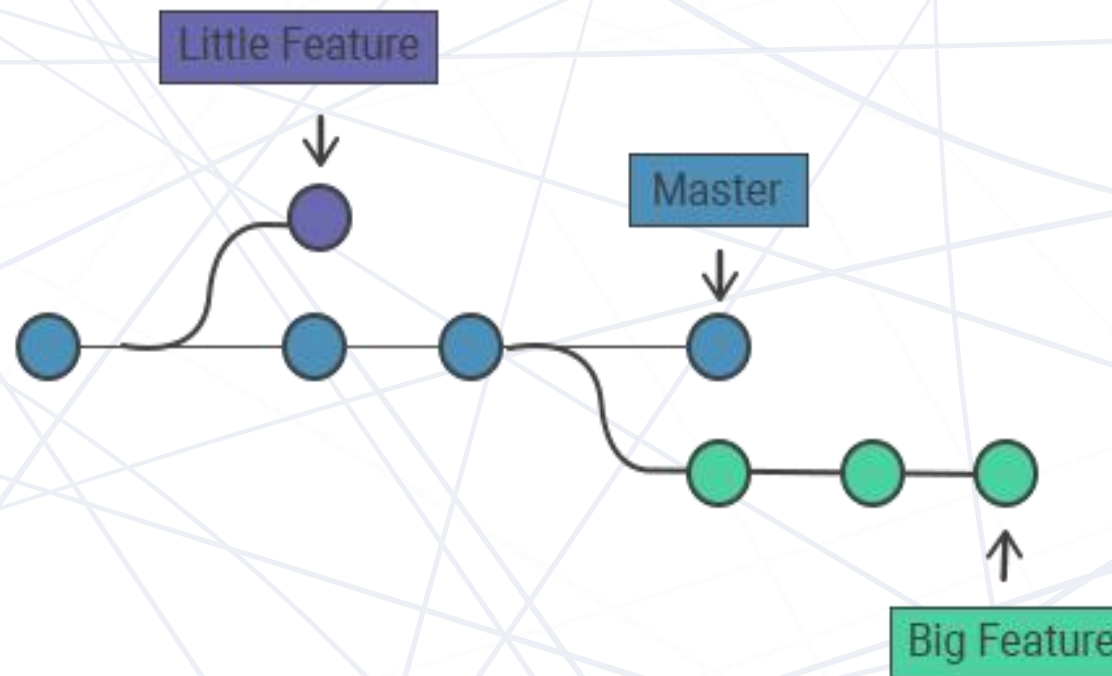
Ciclo:

- Crie branches de funcionalidade (feature branches)
- Trabalhe, Commite e Envie (Push):
- Revise e Discuta
- Mescle e Implante

Github Flow

Fluxo de trabalho





Gerenciar mudanças e histórico

Branch e Remoto